



Institute of Computer Science VII
Robotics and telematics

Master's thesis

Development of a randomized Hough transformation to Cylinder detection in laser scans

Sarah Menninger

January 2021

First supervisor: Prof. Dr. Andreas Nüchter Second
supervisor: Prof. Dr. Klaus Schilling Supervisor: Michael
Bleier

Summary

Besides planes, cylinders are one of the most frequently occurring basic shapes in the industrial sector, as they serve as geometric primitives for piping, scaffolding, and the like.

A fast, efficient, and robust method for detecting cylinders in point clouds is therefore essential to minimize manual steps in model generation. Also

If it is possible to detect cylinders using a 5D Hough transform, then

This approach is very complex and time-consuming. For this reason, this method is based on

Not directly applicable to large point clouds. A reduction in complexity can be achieved by

A sequential two-step approach is enabled. The first step uses a 2D Hough transform to find a strong hypothesis for the cylinder axis orientation.

The second part deals with finding the radius and cylinder position in three-dimensional space. The present work presents a randomized Hough transform.

(RHT) based implementation presented, which deals with the most efficient possible cylinder design

and to automatically detect partial cylinders down to half cylinders. The results are then verified and evaluated using simulation and real-world data.

Table of contents

1 Introduction	1
1.1 Motivation	1
1.2 Objectives and scientific contribution	2
1.3 Structure of the work	2
2 State of the art	5
2.1 Cylinder detection with segmented point clouds.	5
2.2 Cylinder detection on unsorted point clouds.	6
3 Cylinder detection using the Hough transform	7
3.1 Hough transform	7
3.2 2D Hough Transformation	8
3.3 Cylinder detection.	9
3.3.1 Cylinder Parameterization	10
3.3.2 Cylinder axis detection by Hough transformation.	11
3.3.3 3D Hough transform.	12
3.3.4 Position and radius calculation	13
3.3.5 Randomized Hough Transform	14
4 Implementation	17
4.1 Cylinder axis detection	18
4.1.1 Preparation - Calculation of spherical normals.	18
4.1.2 Accumulator	22
4.1.3 RHT Cylinder Shaft Detection	24
4.1.4 Generation of an orthonormal coordinate system.	29
4.2 Position and radius calculation	30
4.2.1 Preparation - Selection of probable cylinder points.	31
4.2.2 Preparation - Projection of the cylinder points.	31
4.2.3 Standard Hough transformation.	33
4.2.4 Randomized Hough transform.	36
4.2.5 Least-Square Circle Fitting.	39
4.2.6 Pseudocode of RHT circuit detection	42
4.3 Cylinder end detection and validation of detected cylinders.	43

4.3.1 Cylinder end detection	43
4.3.2 Detection of duplicate cylinders	44
5 Experiments and Evaluation	47
5.1 Evaluation of the circle detection	48
5.1.1 Evaluation of circle detection with a fault-free cylinder axis and noise-free Point cloud . . 49	
5.1.2 Evaluation of circle detection with an erroneous cylinder axis and a noise-free point cloud . . 54	
5.2 Experiment and evaluation of cylinder detection.	62
5.2.1 Generation of test data and distribution of detected cylinders.	63
5.2.2 Evaluation of cylinder axis detection on a noise-free point cloud	65
5.2.3 Evaluation of circle detection on a noise-free point cloud	68
5.2.4 Evaluation of cylinder detection on a noise-free point cloud	71
5.2.5 Generation of real scan and categorization of the detected cylinders.	72
5.2.6 Evaluation of cylinder axis detection on real data.	75
5.2.7 Evaluation of circle detection on real data.	79
5.2.8 Evaluation of cylinder detection on real data.	82
5.3 Experiment and evaluation of the detection of half cylinders	82
5.3.1 Generation of test data and distribution of detected cylinders.	83
5.3.2 Evaluation of cylinder axis detection on a noise-free point cloud	86
5.3.3 Evaluation of circle detection on a noise-free point cloud	90
5.3.4 Evaluation of cylinder detection for half cylinders.	93
5.3.5 Generation of real scan and categorization of the detected cylinders.	93
5.3.6 Evaluation of cylinder axis detection on real data.	97
5.3.7 Evaluation of circle detection on real data.	101
5.3.8 Evaluation of partial cylinder detection on real data.	106
5.4 Summary evaluation of cylinder detection.	107
6 Summary and Outlook	109

Chapter 1

Introduction

This work deals with the development of a randomized Hough transformation for Cylinder detection in laser scans. A two-part sequential algorithm was used for this purpose. selected which first determines the cylinder axes and then uses them to using circle detection, radius and position of the cylinder in three-dimensional space to determine.

1.1 Motivation

3D laser scanning has become widespread due to its effectiveness and speed.

A way to create dense three-dimensional spatial data, called point clouds.

Since cylinders, along with planes, are one of the most common basic shapes in the industrial sector

Cylinder detection is a fundamental component for generating an accurate model of an industrial scene [22]. Cylinders can serve as graphical primitives for pipe systems.

or tanks are used, but they thus occur in different sizes and orientations. At the same time, many cylinder detection algorithms on unsorted point clouds often place multiple requirements on the cylinder parameters, such as axis orientation.

or the radius size [19]. For this reason, this paper presents an algorithm that can detect cylinders with arbitrary axes and radii.

With this approach, it is possible not only to produce whole cylinders, but also partial cylinders, even half cylinders. robust and quick to detect. At the same time, the algorithm is designed to be as simple as possible.

It is expandable to include additional conditions, which can help minimize runtime and false detection rate, especially in partial cylinder detection.

1.2 Objectives and scientific contribution

The aim of this work is to develop a general cylinder detection algorithm based on a randomized Hough transform, without requiring restrictions on the radius range or the cylinder axis direction. Simultaneously, it should also be possible to successfully detect partial cylinders, down to half-cylinders, and finally, the algorithm should be validated on simulation and real-world data, and any potential weaknesses should be clearly identified.

Such a general algorithm, which does not impose any fundamental requirements on the cylinder parameters, is a valuable scientific contribution, since most works to date deal only with specialized solutions. These are faster, especially for large point clouds, but usually predefine strict conditions for the cylinder parameters, such as predetermined axis directions ([15]) or a fixed radius range ([22], [19]).

In contrast, the randomized Hough transformation approach developed in this work enables the detection of cylinders and subcylinders without requiring any parameter restrictions. The option to restrict the cylinder parameters remains available.

This approach can therefore be easily extended to prevent false detections and optimize its runtime.

The basic idea of cylinder detection using a two-step Hough transform originates from article [22]. However, the actual implementation differs significantly, as [22] has weaknesses in several areas. For example, the radius is not universally valid, and the runtime can be improved in many places.

Furthermore, the algorithm presented in this work offers further developments:

- The evaluation of the Hough space of the cylinder detection is performed using a window-filters (instead of a simple threshold).
- The second step of cylinder detection (circle detection) allows for the use of a parameter-
The trisable validation function is used to specifically detect partial cylinders.
- The result of the circle detection is refined using a least-squares approach.
- The cylinder ends are calculated using the detected lateral surface points.

1.3 Structure of the work

The following work is divided into the main chapters "State of the Art" (2), "Cylinder Detection Using the Hough Transform" (3), "Implementation" (4), "Experiments and Evaluation" (5), and finally, "Summary and Outlook" (6). Chapter "Cylinder Detection Using the Hough Transform" (3) deals with the theoretical basis of the sequential two-part Hough transform based on [22]. The actual development of the randomized Hough transform and its extension are discussed in the following chapters.

through a least-squares approach, is described in the chapter “Implementation”(4), where The most important main steps of the algorithm will be clarified again at the end of the relevant section using pseudocode. This chapter will also address weaknesses. of the algorithm presented in [22] and how the approach presented in this paper this can be minimized. The chapter “Experiments and Evaluation”(5) deals with the Evaluation of the cylinder detection, whereby it is first validated on simulated data and then retested on real data to identify possible weaknesses of the algorithm. to find.

Chapter 2

State of the art

Due to the diverse applications of cylinders, various methods exist today for detecting them in 3D point clouds. Generally, cylinder detection methods can be divided into two classes: the first requires prior segmentation, while the second works directly on the raw point clouds.

2.1 Cylinder detection with segmented point clouds

The first method attempts to fit a cylindrical model to a set of segmented points. This is usually done by minimizing the orthogonal distances to the cylinder surface, for which the least-squares approach can be used [16]. Good initialization parameters are important here to avoid local minima. A disadvantage of this approach is its strong dependence on the quality of the pre-segmentation [24]. Especially with real-world data, the quality can be negatively affected by noise or outliers.

Such an approach can be suitable when automatic cylinder detection is not possible due to numerous outliers, and user input is therefore required for successful detection. For example, an approach was implemented for the 3DTK toolbox [1] which uses a least-squares approach to preselect the best possible cylinder by manually selecting some cylinder points from the point cloud. However, since this approach requires discretization of the cylinder axis and the selected points are usually error-prone, this initial estimate must be further refined. For this purpose, points from the nearest cylinder neighborhood are also considered. Using a RANSAC approach and the previously calculated cylinder axis, it is possible to determine an improved cylinder axis from the new points. This then allows the cylinder radius to be determined even more precisely using a least-squares approach.

As part of this implementation, a display function (qtshow) from 3DTK [1] was extended so that it is also possible to display cylinders in point clouds. This function was implemented in

This work, for example, uses it in Chapter 5 to represent the detected cylinders.

2.2 Cylinder detection on unsorted point clouds

This dependence on pre-segmentation is avoided in the second approach.

RANSAC or Hough transform-based methods can process cylinders without prior transformation.

Segmentation can be extracted from point clouds.

A RANSAC approach uses a random sampling method with as few data points as possible to impose a model onto the data points. Further possible data points are then added as long as necessary. Models are calculated until a sufficiently good solution is found. In the case of cylinder detection, a cylinder model can be calculated from just two measurement points and their surface normals [23]. If conditions for the alignment of the cylinder axes are known,

Furthermore, a RANSAC approach can be used to find cylinders in very large point clouds.

to detect [15]. However, RANSAC can also be used as a partial solution for cylinder search. For example, cylinder axis orientation can be determined using RANSAC via the

The Gaussian sphere can be calculated and then used as input for a least-squares fitting [8]. The major disadvantage of a RANSAC-based approach is the choice of parameters,

which depends heavily on the signal-to-noise ratio. These parameters are, among other things, important for the

The termination of the sampling procedure is responsible. An incorrect parameter choice can increase the probability of a poor result, which in the worst case can mean that

Existing cylinders could not be found.

In contrast to RANSAC, a Hough transform considers all measurement points and uses

These cells accumulate in Hough space. A global maximum.

In Hough space, a solution for a possible model is thus available. A Hough transformation

This approach has the advantage of being stable when there are many existing cylinder instances and outliers. are available. However, a major disadvantage is the computational effort that increases with the number of

Input points n , the number of sampling points of a Hough space dimension s , but before

This is also related to the number of parameters p being sought. The spatial and temporal complexity of a fitting in three-dimensional space can be approximated by

$O(sp)$ or $O(sp \cdot 1 \cdot n)$. Since a cylinder has 5 degrees of freedom, a Hough transform can very quickly reach the limits of the desired computational effort and thus also the time required.

explode. Article [22] provides the basic idea for this using a two-part Hough transformation.

for the detection of cylinders. This idea serves as the basis for the methods developed in this work.

Cylinder detection.

Therefore, in the following chapter "Cylinder detection using the Hough transform"(3)

The underlying theory of this master's thesis, the Hough transformation, the possible division into axis orientation as well as position and radius calculation, and subsequently a possible randomization of the approach were discussed in more detail.

Chapter 3

Cylinder detection using the Hough transformation

3.1 Hough Transformation

The Hough transform is a method for detecting parameterized objects. It is frequently used in two dimensions for line and circle detection. However, it can also be used for plane detection in three dimensions or, as in this case, for cylinder detection.

One disadvantage of the standard Hough transform is the rapidly increasing spatial and temporal complexity with each parameter. A cylinder has 5 degrees of freedom and is represented below by 5 parameters. It should therefore not be detected by a single Hough transform. Instead, it is advisable to use two sequential Hough transforms, the first determining 2 parameters for the cylinder axis and the second determining 3 parameters for the radius and position of the cylinder. [22]

Since the 2D Hough transform for cylinder axis detection already exhibits a higher level of complexity due to the use of spherical normals as input, the following section briefly explains the basic principle of a 2D Hough transform using the standard example of line detection. Therefore, readers familiar with the fundamentals of the Hough transform are advised to skip section "2D Hough Transform" (3.2) and proceed to section "Cylinder Detection" (3.3).

3.2 2D Hough Transform

The basic idea of the Hough transform can be well described using the detection of collinear points. In this example, an unsorted 2D point set $\mathbf{P}$ is used as input, where each point p_i has a Cartesian x - and y -value. For the sake of simplicity, it is assumed that the points lie on a line l_0 or are freely distributed in space.

If collinear points are to be detected using a Hough transform, the mathematical description of a straight line must first be defined. In the Cartesian coordinate system, a straight line is often described by its slope m and its y -intercept t : $y = mx + t$. A disadvantage of this notation is the representation of lines parallel to the y -axis, since in this case the slope is infinite. To be able to find these lines nonetheless, a different representation is more suitable for the Hough transform: the so-called Hessian normal form.

$$r = x \cos \tilde{y} + y \sin \tilde{y} . \quad (3.1)$$

Here, the line is described by its perpendicular distance to the origin r , and $\tilde{y}$ is the angle between r and the x -axis; thus, $\tilde{y}$ lies in the interval $[0, \tilde{y})$. It follows that a line in a coordinate system with the coordinates r and $\tilde{y}$ is represented as a point $(r_0, \tilde{y}_0)$.

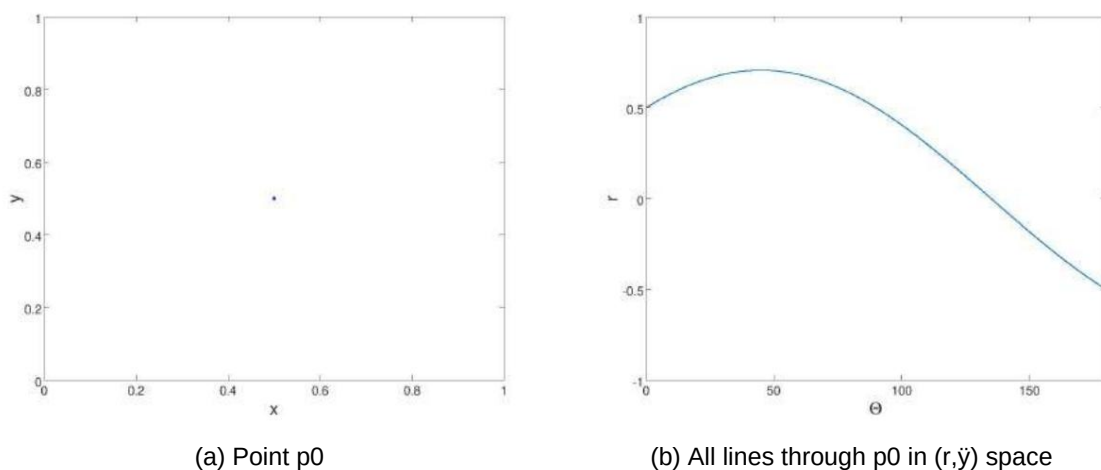
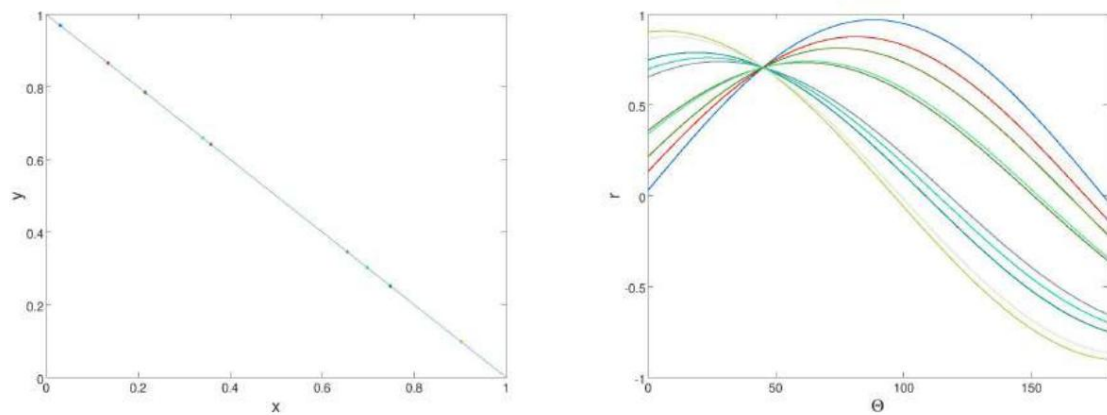


Figure 3.1: Representation of a point p_0 in $(r, \tilde{y})$ space

At the start of the detection, this line, and therefore $\tilde{y}_0$ and r_0 , are still unknown; instead, only the Cartesian coordinates of the points in $\mathbf{P}$ are known. The basic idea of the Hough transform is that a Cartesian input point p_i represents all lines that can pass through p_i . This leads to the conclusion that any point p_i can be represented as a cosine

The function is represented in (r, θ) space (see Fig. 3.1), which includes all possible straight lines through p_i describes.



(a) Random points on a line l_0

(b) All lines through the points on l_0 in (r, θ) space

Figure 3.2: Representation of several points on the line l_0 in (r, θ) space

The Hough transform uses this representation to detect straight lines. For A nice visual representation assumes that l_0 is the slope $m_0 = 1$ and the The y-intercept $t_0 = 1$, or accordingly $\theta_0 = 45^\circ$ and $r_0 \approx 0.7$. There are ten arbitrary points p_i on the line l_0 in the x-axis region $[0, 1]$, with $i = [0, 9]$. If one now plots all lines in (r, θ) space for each point p_i , one obtains ten cosine functions, all of which intersect at the point $\theta = 45^\circ$ and $r \approx 0.7$, since this is the only possible The line is one that can pass through all points p_i (see Fig. 3.2).

In practice, the (r, θ) space is quantized to find the intersection point(s). This quantization divides space into small sub-sections, similar to how individual pixels are divided. in a digital image. The divided sections serve as the basis for the Hough transform. Accumulator cells, which is why the quantized (r, θ) space is also called an accumulator. Each cell has the value 0 at the beginning of the Hough transform. The standard Hough transform calculates, for any input point p_i , all possible (r_i, θ_i) pairs. and increases the corresponding cells of the accumulator by one. The final value of any cell z_i indicates how many points were correct for the corresponding line l_i . Is the final value of z_i If the value was greater than a previously determined threshold, a straight line was detected. [11]

3.3 Cylinder detection

The following sections deal with the theory of the sequential two-step Hough transform for cylinder detection as presented in [22]. For this purpose, First, the parameterization of the cylinders was described in order to then be able to describe

how cylinder detection is possible with a two-dimensional and a further three-dimensional Hough transformation by clever division.

3.3.1 Cylinder Parameterization

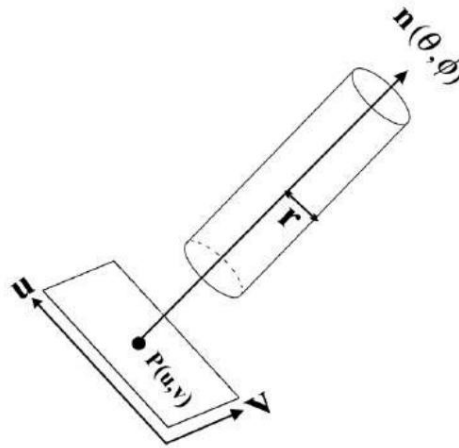


Figure 3.3: Cylinder parameterization from [22]

A cylinder has a total of five degrees of freedom, which can be parameterized in various ways. It is important to note that the Hough transform quickly becomes unwieldy with a large number of parameters. Therefore, the parameters must be chosen so that the cylinder search can be divided into two sequential steps. This division has the advantage of at least partially mitigating the increasing spatial and thus temporal complexity.

In the case of a cylinder, the search can be divided into two sequential steps. In the first step, a strong hypothesis for the cylinder axes can be calculated using a 2D Hough transform. In the second step, a 3D Hough transform is then used to determine a radius and position for each axis found in the first step. This sequential division reduces the maximum number of parameters of a Hough transform to three. [22]

To enable this division, the cylinder is described using 5 parameters. For the first 2D Hough transform, the direction of the cylinder axis from the origin is described using the polar coordinates $\tilde{y}$ and $\tilde{\phi}$. se is defined as:

$$\mathbf{n} = [\cos(\tilde{y}) \sin(\tilde{\phi}), \sin(\tilde{y}) \sin(\tilde{\phi}), \cos(\tilde{\phi})] . \quad (3.2)$$

In order to determine the position and radius in the second step, the cylinder is projected onto a plane along its cylinder axis, using the plane coordinate system.

with axes $\mathbf{u}$ and $\mathbf{v}$ orthonormal to the cylinder axis. Thus, the three-dimensional cylinder becomes a two-dimensional circle with center $\mathbf{cm} = (um, vm)$ and radius r . The equation of the circle is:

$$r^2 = (xp - um)^2 + (yp - vm)^2 \quad (3.3)$$

Where (xp, yp) describes the projected unsorted set of points on the circle's boundary. In reality, however, (xp, yp) contain errors. From this, the parametric equations of the circle follow:

$$\begin{aligned} xp &= um + r \cos(\dot{y}p) \\ yp &= vm + r \sin(\dot{y}p) \end{aligned} \quad (3.4)$$

In the following, this parameterization is used to explain the theory behind a two-stage Hough transformation for cylinder detection, always using publication [22] as a basis.

3.3.2 Cylinder axis detection by Hough transformation

For successful cylinder detection, the first step is to find a strong hypothesis for the cylinder axes.

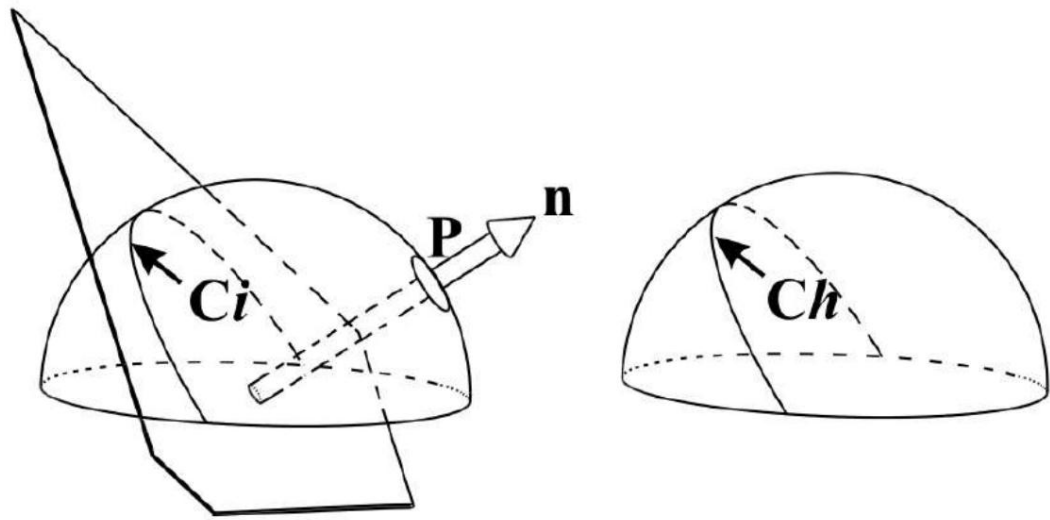
A Hough transform can exploit the special property that the normals of a cylinder form a great circle on the Gaussian sphere. This great circle is formed by the intersection of the Gaussian sphere with a plane of origin whose normal vector is equal to the cylinder axis. Thus, the Hough transform essentially detects a plane of origin that intersects the unit sphere.

The standard Hough transformation to find a plane requires a three-dimensional Hough space for plane detection, where two parameters describe the normal of the plane in spherical coordinates and another parameterizes the distance to the origin.

Since the desired cutting plane is always an origin plane, the distance is always zero. Therefore, the Hough transform for detecting an origin plane has only two parameters. Furthermore, a plane that intersects the Gaussian sphere is sought. This means that each input point votes for a circular region in Hough space. [22]

Figure 3.4 illustrates the voting procedure for point $\mathbf{P}$. The left-hand figure shows the normal $\mathbf{n}$ from the origin to $\mathbf{P}$. The origin plane, also shown, is defined by the coordinate origin and $\mathbf{n}$. The intersection of the origin plane and the hemisphere is labeled $\mathbf{Ci}$. The figure on the right shows the great circle $\mathbf{Ch}$, for which $\mathbf{P}$ votes using the calculated $\mathbf{Ci}$.

To implement such a voting system, the normal to the unit sphere must first be calculated for each input point. This results in a Gaussian sphere calculated from the normals of the point cloud being used as input. However, since the Hough space is also

(a) Point P on the Gaussian input sphere [22](b) P votes for the large circle shown [22]**Figure 3.4:** Representation of the voting procedure for point P (Fig. from [22])

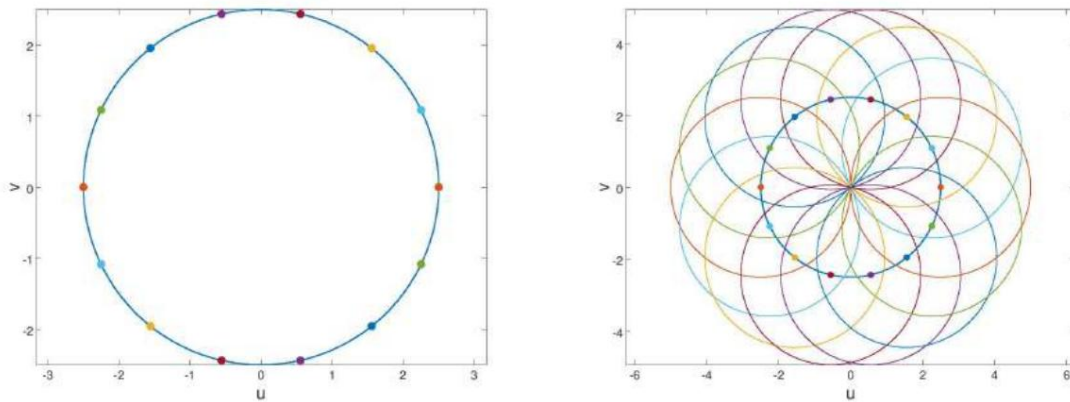
Since it consists only of the orientations of the plane normals, it can also be interpreted as a Gaussian sphere. To avoid confusion, the terms Gaussian input sphere (calculated from the point cloud) or Gaussian Hough sphere (which represents Hough space) will be used in the following.

Each normal, or rather the corresponding point on the Gaussian input sphere, votes for all orthogonal planes on the Gaussian Hough sphere. Each direction thus has an orthogonal set of three-dimensional points that form a plane. The special feature here is that the magnitude of each individual set point is 1. This relationship guarantees that for every intersection plane with the Gaussian Hough sphere, a great circle is formed. Essentially, each input normal votes for the great circle in Hough space that has the same normal vector as the point. [22]

The intersection of all great circles of a cylinder forms a maximum in Hough space and estimates the cylinder axis direction.

3.3.3 3D Hough Transformation

The 2D Hough transform can also be extended to parameterized three-dimensional objects such as circles. The core of the Hough transform remains the accumulator and the detection of its maxima. However, it is important to understand the impact of this extension on complexity. For a three-dimensional fit, the spatial complexity can be approximated by $O(sp)$, where s is the number of samples along a Hough dimension and p is the number of parameters being sought. The corresponding time complexity is $O(sp^2n)$, where n is the number of points. Since it



(a) Circle K with radius 2.5 and a set of points P_k (b) Circle K with a set of points P_k , where a new circle with radius 2.5 is drawn around each point on the circle's boundary with size $n_k = 15$ from P_k

Figure 3.5: Illustration of the basic idea, the standard Hough transform for circle detection

In this specific case, where the evaluation involves 3D point clouds from a laser scanner, n can be on the order of millions of points. This makes the Hough transform very computationally intensive for more than three parameters. [22]

3.3.4 Position and radius calculation

The previously determined hypothesis for the cylinder axis direction can be used to calculate the cylinder's position and radius. If multiple cylinder axes are found in step 1, the position and radius calculations must be repeated for each axis.

At the beginning of the position or radius determination for any detected axis n , all input points must be projected onto a plane along the cylinder axis direction. If the axis direction is perfect, the cylinder will be represented as an exact circle on the plane. To project the points as quickly as possible, an orthonormal coordinate system is calculated, one axis of which is equal to the cylinder axis orientation $\mathbf{n}$. There are two ways to construct this system using the cylinder axis vector: Gram-Schmidt orthonormalization or SVD (Singular Value Decomposition) can be used. Regardless of the technique used, the result is an orthonormal coordinate system described by the three basis vectors $(\mathbf{u}, \mathbf{v}, \mathbf{n})$, where $\mathbf{n}$ is the cylinder axis. These basis vectors can then be used to project the points onto the plane. [22]

Once the two-dimensional projection of each 3D input point has been calculated, the circles of the projected cylinders with axis $\mathbf{n}$ can be detected using a circle-fitting algorithm. Here, [22] proposes implementing a Hough transform according to [13].

The underlying idea is relatively simple: A circle K with radius r_k contains an n_k set of points P_k . If n_k new circles K_n with radius r_k are created, each centered on a point from the set P_k , then all the new circles K_n intersect at the center of K . This is further illustrated in Figure 3.5. The left image shows a circle K with an example radius of 2.5. Fifteen points lie on its boundary. Each of these points serves as the center of a new circle, also with radius 2.5, in the right-hand figure. It can be seen that all the new circles intersect at the original center of K . This fact is used for the Hough transform. For any given radius r , a point in Hough space always votes for a region of a circle with itself as its center. Thus, any point with the projected coordinates (up, vp) in Hough space would vote for all cells described by:

$$(r \cos \tilde{y} + up, r \sin \tilde{y} + vp) . \quad (3.5)$$

The maximum in Hough space thus directly describes the orientation and radius of the cylinder. [22]

A major drawback of this method, however, is that in most cases the radius of the cylinders is unknown. Therefore, r must be sampled in this procedure. This becomes difficult if no maximum or minimum for r is given and the required resolution is unknown. Furthermore, the time complexity increases with each additional sampled radius. For this reason, a randomized Hough transform combined with a least-squares approach was used. Section 3.3.5 describes the general advantages of a randomized Hough transform compared to the standard Hough transform, and the development of randomized circle detection is explained in the "Implementation" chapter (Section 4.2).

In general, any known algorithm can be used for circle detection, as long as all projected cylinders with axis $\mathbf{n}$ are found in the end. Then, the circle's center point simply needs to be transformed back into the world coordinate system. This yields a 3D point on the cylinder's axis, thus determining the cylinder's position.

3.3.5 Randomized Hough Transform

Despite the sequential two-part division, calculating a standard Hough transform can be very computationally intensive, as the input dataset can comprise several hundred thousand points. With a well-chosen accumulator, the Hough space for the first Hough transform can be effectively represented using a quantized half-unit sphere (see Section 4.1.2 for a more detailed description). However, this becomes more difficult in the second step, as the general nature of the approach imposes no simple limits on the radius and position of the cylinder for circle detection. Therefore, it is difficult to construct a suitable Hough space with sufficiently high resolution beforehand for circle detection, since the boundaries of the Hough space, as well as the regions of interest, are difficult to define.

The number of required input points, as well as the problem of discretizing the Hough space, can be reduced using a randomized Hough transform (RHT). The goal of the RHT is to consider only a subset of the input points in order to determine the parameterized object with a certain probability. The RHT uses an accumulator that expands the cells according to the matches, instead of discretizing the Hough space beforehand. This helps, for example, in the case of circle detection, where the accumulator is only expanded where there is a certain probability of a circle being found. [10]

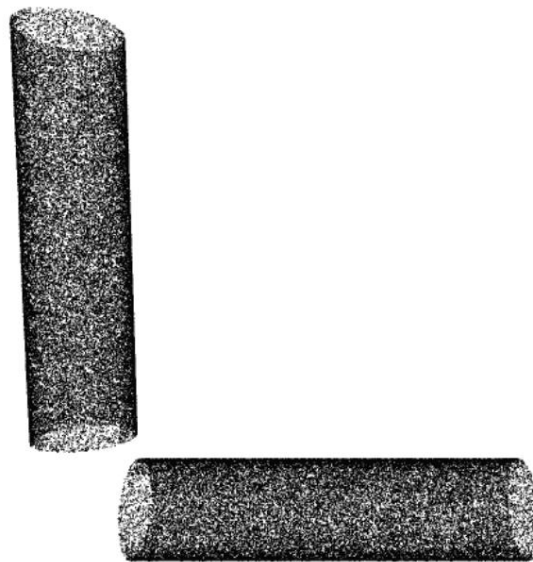
The procedure of the RHT is very similar for the two steps of cylinder detection. Instead of a single point, the RHT selects enough random points to compute the parameterized object. In the case of an origin plane, this is at least two points, and in the case of a circle, at least three. The computed object is then used to accumulate the corresponding value in Hough space. For cylinder axis detection, this means that two randomly selected spherical normal points vote for one cell in the accumulator, while in the standard Hough transform, one spherical normal point votes for a great-circle segment in Hough space. This method allows only a fraction of the input points to be considered, as long as the signal-to-noise ratio is sufficiently high.

Ideally, this method only partially considers the Hough space.

One disadvantage of RHT is the dependence of the parameters on the signal-to-noise ratio. In the special case of cylinder detection, how often n points must be randomly selected and the parameterized object determined depends strongly on the probability of randomly selecting two or three points with the same cylinder axis direction or the same circle, respectively. Thus, in addition to the number of outliers in circle detection, the number of cylinders with the same axis direction also plays a role, since these are all found in the same circle detection. This means there are no universal parameters that can be used for every point cloud; rather, the parameters are highly dependent on the input and must therefore be determined by the user. [10]

Chapter 4

implementation



created by 2016
<http://vireethi.de>

Figure 4.1: Point cloud of two simulated cylinders positioned perpendicular to each other

This chapter deals with the implementation of cylinder detection. The theory from the chapter "Cylinder Detection Using the Hough Transform" (see Fig. 3) is applied, extended, and partially adapted to better cover certain special cases and enable a more efficient implementation. Important steps are summarized and illustrated at the end of each section using pseudocode.

4.1 Cylinder axis detection

To better understand the individual steps of cylinder axis detection, a simple generated example is presented. For this purpose, two perpendicular cylinders with a radius of 2.5 cm and a length of 20 cm were chosen (shown in Figure 4.1). Each cylinder was sampled with 40,000 points. To ensure the best possible representation of the theory, no simulated noise was used in this case.

4.1.1 Preparation - Calculation of spherical normals

An unsorted set of points $\mathbf{S}$ is used as input for cylinder axis detection. However, these points cannot be directly passed to the accumulator because, in the case of cylinder axis detection, an origin plane detection is calculated using the spherical normal. Therefore, preprocessing of the point set $\mathbf{S}$ is necessary.

As a first step, a normal is calculated for each point $\mathbf{p} = (px, py, pz)$ from the set $\mathbf{S}$. For this purpose, [22] proposes using Hoppe's method. This method, based on the least squares principle, allows normals to be calculated even on raw point clouds. The basic idea is to fit a plane as closely as possible to the point and its nearest neighbors, and then to determine a vector perpendicular to the plane. The best possible plane is determined by minimizing the squared errors of the point distances. [12]

Normal calculation

Given a point $\mathbf{p}$ from the set $\mathbf{S}$, with coordinates $(x, y, z)^T$, k nearest neighbors $(\mathbf{p}_i, i=1\dots k)$ are selected for $\mathbf{p}$. These are not determined directly from the unsorted point cloud as in [22], but using a range image created from the points.

We are looking for a tangent plane $\mathbf{T}(\mathbf{p})$ defined by the point $\mathbf{p}$ lying on it and a unit normal vector $\mathbf{n} = (nx, ny, nz)$, with $|\mathbf{n}| = 1$. Therefore:

$$nxx + nyy + nzz - d = 0. \quad (4.1)$$

The least squares method determines $\mathbf{T}(\mathbf{p})$ by minimizing the error:

$$e = \sum_{i=1}^k (\mathbf{p}_i - \mathbf{p})^T \mathbf{n} - d)^2. \quad (4.2)$$

The solution for the minimum error can be determined using the covariance matrix:

$$\mathbf{M} = \frac{1}{k} \sum_{i=1}^k (\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T, \text{ where } \bar{\mathbf{p}} = \frac{1}{k} \sum_{i=1}^k \mathbf{p}_i. \quad (4.3)$$

The smallest eigenvector of the matrix $\mathbf{M}$ corresponds to $\mathbf{n}$. [12]

However, one disadvantage of this method is that the calculation of the matrix $\mathbf{M}$ and its smallest Eigenvectors are comparatively computationally intensive and must be executed for each input point. Furthermore, the traditional approach depends on k .

One way to calculate the normals without the dependence on k is, for example, by the The normalized least squares approach can be used. The idea behind this approach is that Noise is not considered as independent and identically distributed as in the traditional approach. is, but the noise is largely caused by a distance measurement error, which propagates linearly in the direction of view. Therefore, instead of directly calculating the eigenvectors from $\mathbf{M}$ to To determine, in the case of such an error, it is more sensible to determine the coordinates of the 3D input points. Normalize so that the three principal axes of $\mathbf{M}$ are equal to the unit length. Then The matrix $\mathbf{M}$ is normalized using Cholesky factorization:

$$\mathbf{M} = \mathbf{K} \mathbf{\tilde{Y}} \mathbf{1} \mathbf{M} \mathbf{K}^T. \quad (4.4)$$

$\mathbf{K}$ is a lower triangular matrix for which the following holds:

$$\mathbf{\tilde{Y}} = \sum_{i=1}^k \mathbf{p}_i \mathbf{p}_i^T = \mathbf{K} \mathbf{K}^T. \quad (4.5)$$

The solution is then calculated using the smallest eigenvalue of $\mathbf{M}$. The Cholesky method Factorization does not depend on the number of neighbors k , however, this is possible with a significantly greater computational effort. [6]

Using an approximation, the normal calculation can be accelerated, whereby this This approach is naturally less accurate than the traditional approach. First, the error function must be divided by d^2 :

$$e = \sum_{i=1}^k (\mathbf{p}_i^T \tilde{\mathbf{n}} - d)^2, \quad (4.6)$$

where $\tilde{\mathbf{n}}$ is the desired normal vector of the plane multiplied by a scaling factor. However, the division of d^2 leads to degeneration for $d = 0$, i.e., for all origin levels. For this reason, the k nearest neighbors are not directly matched to the point cloud.

Instead of searching, a range image created from the point cloud is used. This has the advantage that that d is never 0. Thus, for $\tilde{\mathbf{n}}$:

$$\tilde{\mathbf{n}} = \mathbf{M}^{-1} \mathbf{\tilde{Y}} \mathbf{1} \mathbf{b}, \quad (4.7)$$

where $\mathbf{M}^{-1} = \mathbf{\tilde{Y}}^{-1} \sum_{i=1}^k \mathbf{p}_i \mathbf{p}_i^T$ and $\mathbf{b} = \sum_{i=1}^k \mathbf{p}_i$. Subsequently, $\mathbf{n}$ can be normalized by $\tilde{\mathbf{n}}$ The calculation is: $\mathbf{n} = \tilde{\mathbf{n}} / \|\tilde{\mathbf{n}}\|$. This notation, although not optimal in theory, can be used in This offers practical advantages, as both $\mathbf{M}^{-1}$ and $\mathbf{b}$ can be calculated efficiently. However, using an approximation, it is possible to simplify the error function so that The matrix $\mathbf{M}^{-1} \mathbf{\tilde{Y}} \mathbf{1}$ does not need to be calculated every time. Using an extension of r^2 on The right side of the error equation yields:

$$e = \sum_{i=1}^k r_i^2 (\mathbf{v}_i^T \tilde{\mathbf{n}} - r_i)^2 \quad (4.8)$$

with:

$$\mathbf{v}_i = \frac{\ddot{y} \cos \ddot{y}_i \sin \ddot{y}_i}{\sin \ddot{y}_i \sin \ddot{y}_i} \cdot \frac{\ddot{y}}{\cos \ddot{y}_i} \quad (4.9)$$

This results in the following solution:

$$\hat{\mathbf{n}} = \mathbf{M}^{\hat{\mathbf{y}}} \mathbf{1} \mathbf{b}^{\hat{\mathbf{y}}} \quad (4.10)$$

where $\mathbf{M}^{\hat{\mathbf{y}}} = \sum_{i=1}^k \mathbf{v}_i \mathbf{v}_i^T$ and $\mathbf{b}^{\hat{\mathbf{y}}} = \sum_{i=1}^k \mathbf{v}_i / r_i$. The special thing about this approximate form is that the matrix $\mathbf{M}^{\hat{\mathbf{y}}}$ depends only on the image parameters of the range image and thus can be calculated in advance. [6]

In addition to the method of least squares, which is originally used in [22],

Can one also create a "Spherical Range Image" (SRI) by creating a range image?

Use the differential approach. Both approaches exploit the structure of the range images to...

To determine surface normals faster than directly from the raw data.

If the position(s) of the scanner in the room are unknown, then

After successful calculation, it is difficult to determine the surface normals.

to align. This results in the normals being perpendicular to the cylinder surface.

They can be positioned, but can point away from the cylinder as well as into it. An alignment of the

However, normals are not necessary for successful cylinder detection; this will be explained in more detail in the following sections 4.1.1 and 4.1.3.

Transformation to the unit sphere

Once a normal $\mathbf{n}$ has been determined for each point $\mathbf{p}$, these can be used to

to calculate the rotation matrix $\mathbf{R}$, which then becomes the input normal of the Hough

Transformation onto the unit sphere rotates. A great circle on the xy-plane with the z-axis

The normal corresponds to:

$$x = \cos(t) \quad y = \sin(t) \quad z = 0, \text{ where } 0 \leq t \leq 2\pi. \quad (4.11)$$

Each point given by equation 4.11 must now be rotated so that the z-axis of the

This corresponds to the normal of the appropriate great circle. Since all points lie in one plane,

For this purpose, the reflection matrix given by the Householder transformation was used.

become:

$$\mathbf{R} = \mathbf{I} - 2\mathbf{b}\mathbf{b}^T \quad (4.12)$$

$$\text{with } \mathbf{b} = \frac{(\mathbf{z} - \mathbf{n})}{\|\mathbf{z} - \mathbf{n}\|} \quad (4.13)$$

where $\mathbf{z}$ and $\mathbf{n}$ are defined as:

$$\begin{aligned} \mathbf{z} &= (0 \ 0 \ 1)^T \\ \mathbf{n} &= (\cos \ddot{y} \sin \ddot{y} \sin \ddot{y} \sin \ddot{y} \cos \ddot{y})^T \end{aligned} \quad (4.14)$$

The input points given by the normals can then be calculated using the rotation matrix $\mathbf{R}$. They are rotated so that they form great circles. These can be calculated using origin plane detection, in this case an RHT, where the cylinder axes are defined by the Plane normals are represented. [22]

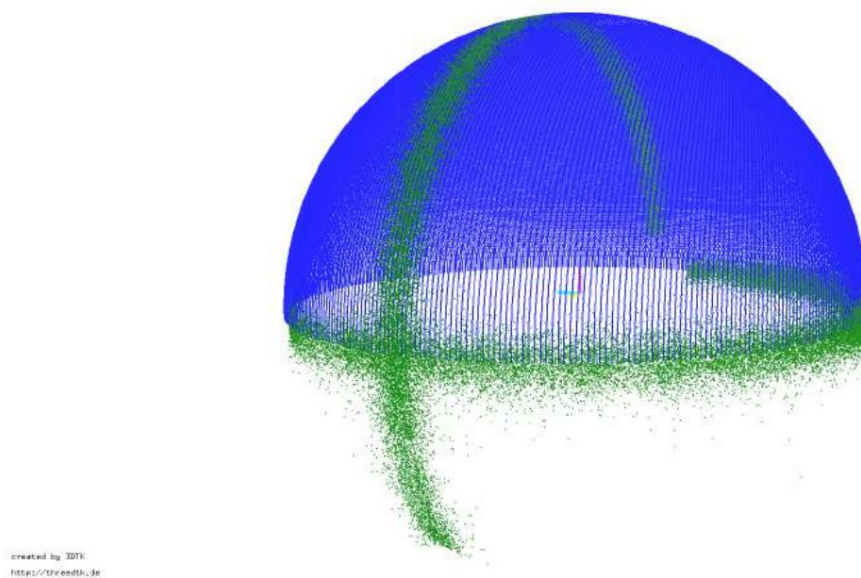


Figure 4.2: Representation of the spherical cylinder normals of two simulated cylinders; blue: representation of sampled unit sphere, green: representation of spherical cylinder normals

Figure 4.2 shows the normals of the cylinders rotated appropriately so that a great circle is formed on the Gaussian sphere for each cylinder axis. The special feature here is that Due to the misaligned normals, instead of a full great circle, there is a half great circle. for a cylinder axis, where the number of points of the half-great circle is equal to the The number of points in the whole would correspond to [missing information]. The reason for this is that half of the cylinder normals are inverted, as they could not be properly aligned due to the missing laser scanner position. However, this does not pose a problem for the subsequent Hough transform. This poses a problem, since all normals still lie on the great circle and are therefore not suitable for the Cylinder axis voting is possible.

The preparation of the unsorted point cloud necessary for cylinder axis detection is summarized in the following pseudocode (1).

Pseudocode of the preparation

Algorithm 1 Cylinder axis detection - Calculation of the spherical normal **Requirement:** Input: Point set **S**

1: Calculate the normal for each point p_i in

S normi 2: Vector $\mathbf{z} = (0 \ 0 \ 1)$

3: Matrix $\mathbf{b}[\mathbf{S.size()}, 3]$ 4: **for** i

$= 0; i < \mathbf{S.size()}; i++$ **do** $[\tilde{y}_i, \tilde{y}_i] = \text{calculate}$

5: sphere coordinates (normi)

6: Vector $\mathbf{ni}[3]$ $\mathbf{ni}[0] = \cos(\tilde{y}_i)$

7: $\sin(\tilde{y}_i) \mathbf{ni}[1] = \sin(\tilde{y}_i) \sin(\tilde{y}_i) \mathbf{ni}[2] =$

8: $\cos(\tilde{y}_i) \mathbf{b}[i, :] = \mathbf{z} \tilde{y} \mathbf{ni}$ 10: 11: **end**

9: **for** 12:

3] **sphericalNormals** $[\mathbf{S.size()},$

13: **for** $i = 0; i <$

$\mathbf{S.size()}; i++$ **do** $\mathbf{b}[i, :]$

14: Matrix $\mathbf{R} = \text{IdentityMatrix}(3) \tilde{y} \mathbf{2} \mathbf{b}$ 15: $\mathbf{R} = (\mathbf{b} \begin{smallmatrix} i \\ j \end{smallmatrix})^T$

sphericalNormals $[\mathbf{i}, :] = \mathbf{R} \tilde{y} \mathbf{normi}$ 16: **end for** 17: **return**
sphericalNormals

4.1.2 Accumulator

Before the actual Hough transform can be computed, the accumulator must first be created. In the case of cylinder axis detection, an accumulator is needed that can vote for the normal of a great circle or, in this specific implementation case, for the normal of an origin plane intersected by half a unit sphere. In the case of [22], the Hough space is represented by a hemisphere that is sampled approximately uniformly [17].

In contrast, a battery that builds up over time is usually used for a RHT (Real-Time Hotlight).

However, since it is possible to represent the entire Hough space using a quantized half-unit sphere for cylinder axis detection, this is omitted in this case. Therefore, the accumulator is generated beforehand as in [22]. It also has the structure of a half-unit sphere, but is created using a different sampling approach.

The following section explains the division of the accumulator in general terms, using a sphere as an example. The key idea behind sampling is that each accumulator cell on the sphere's surface should have the same area. To achieve this, the sphere is divided using latitude ($\tilde{y}$) and longitude ($\tilde{y}$). The unique aspect is that the resolution of each cell is adjusted according to its position on the sphere. The resolution of the longitude $\tilde{y}$ is constant and must be chosen appropriately depending on the problem. Thus, $\tilde{y}$ determines the distance between

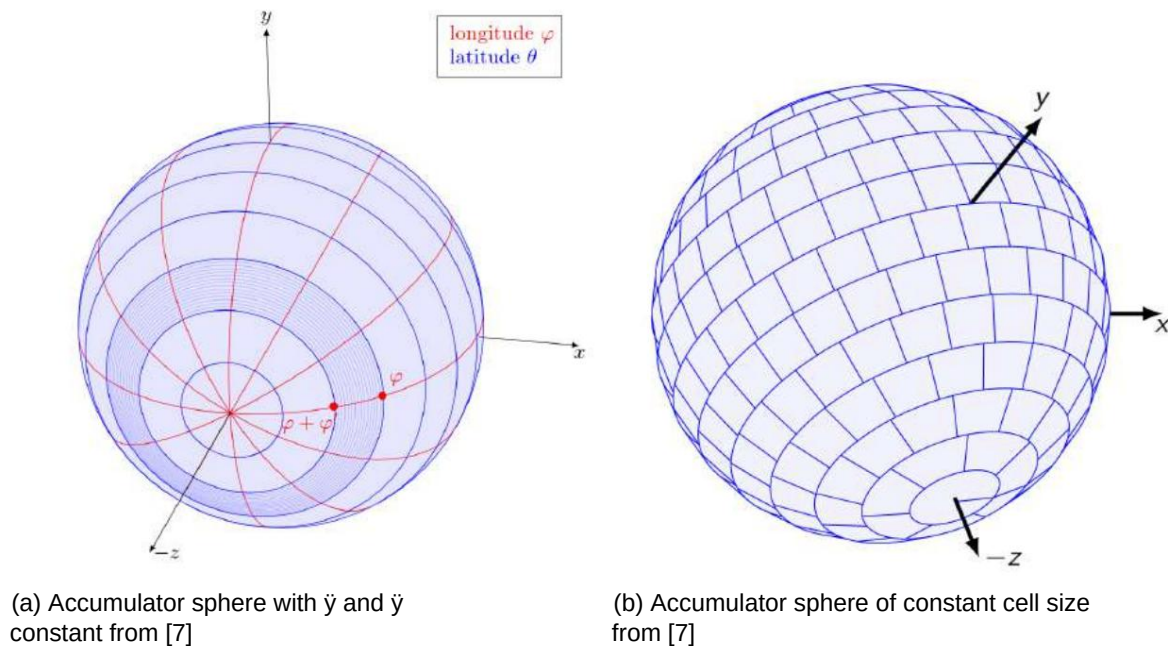


Figure 4.3: Spherical accumulators

The individual meridians, or in other words, $\ddot{y}$ describes how wide the individual circle segments are. The resolution of the latitude must be determined to match the respective longitude so that the subdivision of the circle's area results in cells of equal size. A suitable discretization for the step size $\ddot{y}$ can be determined, for example, from the area of the longitude at $\ddot{y}_i$. The largest possible circle is located at the equator ($\ddot{y} = 0$) and, due to the unit sphere, has a maximum length of $l_{maxi} = 2\ddot{y}$. The length of the circle above $\ddot{y}_i$ is given by $l_i = 2\ddot{y}(\ddot{y}_i + \ddot{y})$. The step size for $\ddot{y}$ can thus be calculated using:

$$\ddot{y}_{\ddot{y}_i} = \frac{2 \cdot \ddot{y} \cdot l_{maxi}}{l_i \cdot N\ddot{y}}, \quad (4.15)$$

where $N\ddot{y}$ corresponds to the number of cells. With such a discretization adapted for the step size $\ddot{y}$, it is possible for the accumulator cells to all have an almost identical area. [7]

Figure 4.3 shows the accumulator sphere. On the left, a sphere can be seen whose latitude and longitude resolutions are the same, resulting in different areas of the subdivided cells. On the right, however, the latitude resolution is adjusted to match the longitude, so that cells of equal size are created. [7]

In the case of cylinder axis detection, only half a unit sphere is needed, since any vector $\mathbf{w}$ and $\mathbf{\ddot{y}w}$ correspond to the same cylinder axis. Therefore, analogous to the sphere sampling described above, half a sphere is sampled and used as an accumulator. This prevents two maxima from being possible in the accumulator for each cylinder axis. Only at

At the edge of the hemisphere, i.e. at the equator of the sphere, cylinder axes have two maxima in Hough space, which are always opposite each other.

4.1.3 RHT Cylinder Shaft Detection

For cylinder axis detection, a standard Hough transform is used in [22] to detect the great circles or the circles of intersection of an origin plane with the unit sphere. Since this serves as the basis for the RHT, the standard Hough transform for origin plane detection is first briefly described, and the necessary modifications for the RHT used in this work are explained based on this.

Description of the origin plane

A plane in three-dimensional space is often described by its directed distance to the Coordinate origin $\check{y}$ (support vector), as well as by its two direction vectors mx and my :

$$\mathbf{z} = mx \cdot x + my \cdot y + \check{y}. \quad (4.16)$$

One disadvantage of this representation becomes apparent when representing vertical planes. In this case, the direction vectors approach infinity. For this reason, another, also frequently used, representation is employed. Let $\mathbf{p}$ be a point on the plane, $\mathbf{n}$ the normal vector orthogonal to the plane, and $\check{y}$ the distance to the origin; then the following results:

$$\check{y} = \mathbf{p} \cdot \mathbf{n} = pxnx + pyny + pznz. \quad (4.17)$$

Using the angles of the normal vector and the coordinate system, it is therefore possible to determine the Factoring the coordinates of $\mathbf{n}$:

$$px \cos \check{y} \sin \check{y} + py \sin \check{y} \sin \check{y} + pz \cos \check{y} = \check{y}, \quad (4.18)$$

where $\check{y}$ describes the angle of the normal vector on the xy-plane and $\check{y}$ describes the angle between the xy-plane and the normal vector in the z direction. [7]

Since, in the case of cylinder axis detection, all the planes sought are always origin planes, the distance to the origin is always zero, thus $\check{y} = 0$. The two-dimensional Hough space is therefore defined by $(\check{y}, \check{y})$, where each discretized point of the Hough space corresponds to an origin plane.

Standard Hough transform

For the standard Hough transform proposed in [22], this means that a Hough transform is computed for each point in the rotated spherical normal set.

Consider, for example, an arbitrary point $\mathbf{p}$ on the Gaussian input sphere. In a standard Hough transform, all accumulator cells that contribute to a

The plane of origin corresponds to the plane on which $\mathbf{p}$ can lie. This leads to a 2D sinusoidal plane. Curve. If two curves from the arbitrary points p_i and p_j intersect, then the curve describes the intersecting curve. corresponding interface in 2D Hough space, the origin plane, which is defined by the two Points p_i and p_j are spanned.

Now, let us consider the set of points P , which lies on the Gaussian input sphere. All points $P_i \in P$ are transformed into Hough space. The more curves are in the accumulator cell, the more The more points intersect $h_j \in (\tilde{y}, \tilde{y})$, the more points lie on the plane of origin, represented by h_j . With each curve, the probability increases that the found origin plane, represented by h_j , corresponds to a sought-after great circle and thus describes a cylinder axis. After all points have been voted on, the most likely cylinder axes can be selected. This will be described in more detail in section 4.1.3. In pseudocode 2, the standard Hough transformation for detecting cylinder axes summarized again briefly. [7]

Although a sequential two-part standard Hough transform is used for cylinder detection in [22], it is significantly less space- or time-complex compared to a 5D Hough transform, but it still has some disadvantages.

Firstly, the number of accumulator cells to be incremented per input point is very large due to the standard Hough transform. Secondly, it may well be that that it is not necessary to consider all input points for cylinder axis detection. From For this reason, a Randomized Hough Transform (RHT) was used for this implementation. used. This has several advantages, which are mainly reflected in the runtime, however It also has one major disadvantage compared to the standard Hough transform: it is non-deterministic.

Algorithm 2 Cylinder axis detection - Standard Hough transform

Require: Input: Spherical Normals Set P

```

1: A = createAccumulator()
2: for every point  $p_i \in P$  do
3:   for each cell  $(\tilde{y}, \phi_i)$  in A do
4:     if  $p_i$  lies on the origin plane  $(\tilde{y}, \phi_i)$  then
5:       A.incrementCell( $\tilde{y}, \tilde{y}$ )
6:     end if
7:   end for
8: end for
9: A.FilterWindowFunction(WindowSize)
10: A.sortRemainingMaximumsByProbability()
11: return A.getMaxima() >  $t_{min}$ 
```

RHT

The Randomized Hough Transform (RHT) has the significant advantage of requiring considerably fewer cells of Hough space to be touched. To achieve this, the RHT exploits the condition that a curve with n parameters is defined by n points. In the case of cylinder axis detection or an origin plane, this means that two input points are always mapped to a single point in Hough space. This point describes the origin plane spanned by the two points and the origin, with the plane normal defining the direction of the cylinder axis.

In contrast to the standard Hough transformation, the RHT proceeds as follows: At the beginning of each step, two random points p_i and p_k are selected. Since an origin plane is sought, a third point p_0 , defined by the origin of the coordinate system, is also given, where this is defined as $(0\ 0\ 0)^T$. From these three points, the plane is then calculated, where the following holds:

$$\begin{aligned} 0 &= \mathbf{n} \cdot p_i \\ 0 &= ((\mathbf{p}_k - \mathbf{p}_0) \times (\mathbf{p}_i - \mathbf{p}_0)) \cdot p_i \\ &= (\mathbf{p}_k \times p_i) \cdot p_i \end{aligned} \quad (4.19)$$

Subsequently, $\tilde{y}$ and $\tilde{y}$ of the origin plane spanned by p_i and p_k (described in 4.1.3) can be calculated, and the corresponding cell in the accumulator $A(\tilde{y}, \tilde{y})$ can be incremented. If a cylinder with axis direction $(\tilde{y}, \tilde{y})$ is present in the scene, a maximum will be observed at the corresponding accumulator cell $(\tilde{y}, \tilde{y})$ after a certain number of iterations, since many points lie on the lateral surface of the cylinder, thus increasing the probability of selecting two spherical normals that match $(\tilde{y}, \tilde{y})$. This results in a maximum in the accumulator at the cell corresponding to the cylinder axis. [7]

In plane detection, it is also possible to impose a distance condition on the two randomly selected points. If the distance between two of the three points is too large, they often do not lie on the same plane. For this reason, the distance criterion for plane detection can be defined as:
 $\text{distmax}(p_1, p_2, p_3) \leq \text{tdist}$ [7]

This is not possible in the case of cylinder axis detection, since all rotated spherical normal points of a cylinder lie on a great circle of the unit sphere. Because a great circle is the largest possible circle on a sphere's surface, the distance between two points opposite each other on the great circle is maximal. This means that one cannot determine whether $\tilde{y}$ and p_k are likely to belong to a cylinder axis based on their distance.

However, to counteract errors caused by sensor noise, it may be useful to define a minimum distance criterion such that $\text{dist}(p_i, p_k) > \text{distmin}$.

Figure 4.4 shows the accumulator after a successful RHT. The input used was the two great circles of the mutually perpendicular cylinders (see Fig. 4.4). The blue dots represent cells that were not incremented, while red indicates cells with many accumulations. It is noteworthy that not only two, but three maxima are shown in Hough space. If the accumulator were not defined by just one

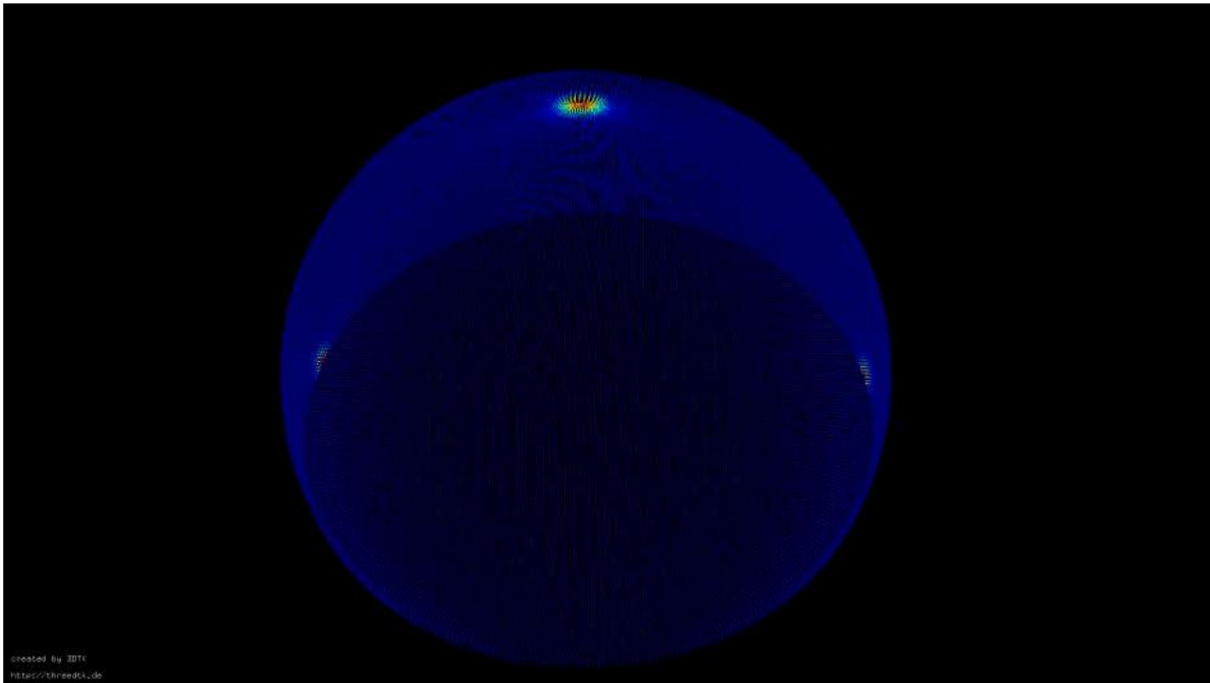


Figure 4.4: Representation of the accumulator along the cylinder axis RHT, where blue represents unincremented and red represents the maximum number of accumulations.

If the cylinder axis were represented not by a hemisphere, but by a unit sphere, a total of four maxima would be detected. This is because one of the represented cylinder axes lies at the edge of the hemisphere, and thus its inverted axis is also represented in the accumulator. These possible double maxima in Hough space cannot be completely neglected for the next steps, as a robust cylinder axis hypothesis is needed to reliably detect cylinders.

Sliding Window Function

After the Hough transform, the most probable cylinder axes must be selected from the accumulator. In [22], a simple threshold is used for this purpose. All accumulator cells whose vote count is greater than the threshold are considered as hypotheses for the cylinder axis direction. This procedure is relatively simple and fast, but can lead to some problems. For example, if a laser scan contains many cylinders with the same axis ni and only one cylinder with a different axis direction nk , where the number of sampled points on each cylinder is the same, then significantly more spherical normal points will vote for ni than for nk . However, due to the discretization of the Hough space, noise on the input points, and inaccuracies in determining the normals, especially at the cylinder edges, the points of an arbitrary cylinder axis will vote not only for one accumulator cell, but for several. To select both cylinder axes nk and ni ,

To select using only one threshold, the threshold must be small enough that at least one cell corresponding to nk has a higher score than the threshold. However, this can in turn lead to the threshold for the ni axis being too small, so that not only is one accumulator cell selected for ni , but all cells in the immediate vicinity of the actual maximum are also larger than the threshold.

To avoid this problem, it is advisable not only to search for the cells with the maximum score, but always to consider the maximum of a narrow region of the accumulator. One possibility for this is the "sliding window" procedure for peak detection [14]. Here, a small two-dimensional window is defined that can overlay the entire peak spread. The most prominent layer corresponds to the center of the square in the Hough space containing the maximum peak. The individual steps of the RHT are briefly described again as pseudocode in Figure 3.

Pseudocode of RHT cylinder axis detection

Algorithm 3 Cylinder axis detection - RHT

Require: Input: Spherical Normals Set P

```

1:  $A = \text{createAccumulator}()$  2: while  $t < t_{max}$  do
3: Choose  $p_1, p_2 \in P$  randomly.  $\text{dist} =$ 
4:    $\text{calculateDistanceBetween2Points}(p_1, p_2)$  if  $\text{dist} < \text{Threshold}$  then
5:
6:   continue
7: end if
8:    $[\tilde{y}, \tilde{y}] = \text{calculateOriginPlane}(p_1, p_2)$  if not
9:    $\text{liesOnAccumulatorHemibulary}(\tilde{y})$  then  $[\tilde{y}, \tilde{y}] =$ 
10:     $\text{invertedPlaneNormal}(\tilde{y}, \tilde{y})$ 
11: end if
12:  $A.\text{increment}(\tilde{y}, \tilde{y})$ 
13:  $t++$  14:
end while
15: Cylinder axes  $z = \text{SlidingWindowFilter}(A, \text{windowSize})$  16:  $z =$ 
 $\text{sortAxesByProbability}(z)$ 
17: return  $z$ 
```

4.1.4 Generation of an orthonormal coordinate system

After the evaluation of the Hough space has been completed, the selected cylinder axes are

The data is passed individually to the circle detection. Before the transfer, a [missing information] must be entered for the corresponding axis.

An orthonormal coordinate system must be determined so that circle detection is as fast as possible.

It can project the 3D points along the cylinder axis onto a plane. The following applies to the three-dimensional orthonormal basis vectors $\mathbf{u}$, $\mathbf{v}$ and $\mathbf{w}$:

$$\|\mathbf{u}\| = \|\mathbf{v}\| = \|\mathbf{w}\| = 1 \quad (4.20)$$

and

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{w} = \mathbf{w} \cdot \mathbf{u} = 0. \quad (4.21)$$

For a right-handed base, the following applies:

$$\mathbf{w} = \mathbf{u} \times \mathbf{v}. \quad [18] \quad (4.22)$$

Since the orthonormal coordinate system in this case is used for a projection of the point cloud

If the coordinate system is to be used along the cylinder axis, it is essential that the direction of the cylinder axis corresponds to an orthonormal basis vector. Two possible implementations for calculating this coordinate system are proposed in [22]. Firstly, the

Gram-Schmidt orthogonalization, and secondly a solution using an SVD (singular value decomposition) proposed.

For both methods, N nearest neighboring points for the cylinder axis must first be found.

[22] will be, using an approximate uniform sampling approach for this purpose. From these

Given N vectors, an orthonormal basis is to be constructed. The classical Gram-Schmidt

The approach starts with a vector and expands the subspace by one vector in each step.

one dimension. Numerically speaking, however, this approach is poor because the rounding errors add up, which is why at least a modified Gram-Schmidt algorithm is needed.

should be used (described in [9]). For the second solution, however, it is necessary to use

An $M \times N$ matrix A was formed from the N 3-dimensional vectors and this was processed using an SVD.

can be solved [21].

Both methods have the major disadvantage of being comparatively computationally intensive for the relatively small sample size.

These are simple problems of calculating an orthonormal coordinate system for a cylinder axis. This is especially true when considering the use of RHT cylinder axis detection.

Only a strong hypothesis was calculated, not the exact cylinder axes. This can

This can lead to incorrect or duplicate cylinder axes being present alongside the correct ones, due to...

Noise, outliers, and calculation inaccuracies in the spherical normal calculation must be found. Therefore, the SVD must ideally be performed at least once for all

Cylinder axis directions, however, are calculated much more frequently in reality.

For this reason, a different approach was used for the implementation presented here.

used. This does not include the neighbors of a maximum, but calculates

the coordinate system directly from the axis, which is why it makes sense to determine the resolution of the

to choose accumulators more precisely and to determine maxima not with a simple threshold, but to detect with a sliding window filter. A finer resolution of the accumulator is possible due to RHT, because the cells to be accumulated are randomly selected for two points change, which is not the case with the standard Hough transform.

Thus, it is possible to derive the orthonormal coordinate system directly from the cylinder axis direction $\mathbf{n}$. Since the cylinder axis direction has a length of 1 due to the unit sphere of Hough space, the following holds:

$$\mathbf{w} = \frac{\mathbf{n}}{\|\mathbf{n}\|} = \mathbf{n}. \quad (4.23)$$

Next, an arbitrary vector $\mathbf{t}$ is defined that is not collinear with $\mathbf{w}$. Using these vectors and the cross product, it is possible to find the unit vector $\mathbf{u}$ perpendicular to $\mathbf{w}$ to determine:

$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|}. \quad (4.24)$$

If $\mathbf{t}$ were collinear with $\mathbf{w}$, the denominator would be 0 and the equation would be undefined. From this Basically, $\mathbf{t}$ should not be almost collinear with $\mathbf{w}$, otherwise the result will have poor accuracy. Subsequently, $\mathbf{v}$ can be generated from $\mathbf{w}$ and $\mathbf{u}$ using the cross product:

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}. \quad (4.25)$$

This approach makes it very easy and without significant computational effort to use the to establish an orthonormal coordinate system around the cylinder axis. [18]

4.2 Position and radius calculation

Section 4.1 described how to formulate a strong hypothesis for the direction of a arbitrary cylinder axis. Thus, with this step, two of the five sought were found. Cylinder parameters are determined. The three missing parameters describe the position and the Radius of the cylinder. You must use the second Hough transform for each The found cylinder axis must be redefined and recalculated. For this reason, the Hough transformation for determining radius and cylinder position is significantly more common They are calculated as those for cylinder axis detection.

The second Hough transformation for a cylinder axis direction $\mathbf{n}$ proceeds as follows. Beforehand: First, all possible cylinder points along $\mathbf{n}$ are projected onto a plane. This creates A cylinder is transformed into a circle. This circle can then be found using a Hough transform. The circle's radius corresponds to the cylinder's radius. If the If the center of the circle is transformed back into three-dimensional space, then the The position of the cylinder can be obtained from this.

The following describes the individual implementation steps for the cylinder position as well as the Radius calculation described in more detail.

4.2.1 Preparation - Selection of probable cylinder points

At the beginning of the position and radius determination, it is important to pre-sort the points of the laser scan. Only points whose spherical normals lie on the great circle perpendicular to the cylinder axis direction may be passed to the second Hough transform.

However, due to noise and minor errors in the normal calculation, very few spherical normal points lie exactly on the great circle plane.

For this reason, it is useful to define a threshold value that limits the maximum distance to the great circle plane. This threshold should be large enough to select most of the error-prone spherical normal points that vote for the plane, while simultaneously being small enough to distinguish between the two most similar great circles. Since both the measurement noise and the minimum angular difference between two cylinder axes to be found vary from scan to scan, this parameter must be chosen appropriately for each dataset. For the experiments and analyses in this work, the threshold was set to a maximum distance of 0.01.

The distance d of a spherical normal $\mathbf{pn}$ to a great circle or to a plane of origin perpendicular to the cylinder axis $\mathbf{n}$ is calculated as follows:

$$d = \frac{|\mathbf{n} \cdot \mathbf{pn}|}{|\mathbf{n}|} = |\mathbf{n} \cdot \mathbf{pn}| \cdot |\mathbf{n}| \quad (4.26)$$

Since the cylinder axis was calculated using the unit sphere, it is already normalized, so there is no need to divide by its magnitude when calculating distances. If d is less than the threshold value, the 3D point $\mathbf{p}$ with the spherical normal $\mathbf{pn}$ is used for circle detection; if it is greater, it is discarded for this particular cylinder axis.

4.2.2 Preparation - Projection of the cylinder points

All three-dimensional points whose spherical normals vote for the cylinder axis in the first step are subsequently grouped together in the set P_c . Every point $\mathbf{p}_i \in P_c$ must be projected onto the plane along the cylinder axis before the Hough transformation.

For the most efficient projection possible, an orthonormal coordinate system is used, the setup of which was described in section 4.1.4. It is calculated directly after the cylinder axis detection and passed on to the circle detection.

Let $\mathbf{u}$, $\mathbf{v}$, $\mathbf{w}$ be the orthonormal basis of the cylinder axis coordinate system, where $\mathbf{w}$ is equal to the cylinder axis direction $\mathbf{n}$. Furthermore, let $\mathbf{p}$ be an arbitrary point from P_c . Then the corresponding projected point $\mathbf{pproj}$ can be calculated as follows:

$$\mathbf{pproj} = (\mathbf{p} \cdot \mathbf{u})\mathbf{u} + (\mathbf{p} \cdot \mathbf{v})\mathbf{v} \quad (4.27)$$

For the subsequent Hough transformation, it is useful to convert the still three-dimensional Cartesian coordinates $\mathbf{p} = (px, py, pz)$ into two-dimensional plane coordinates $\mathbf{p2D} = (pu, pv)$ to convert:

$$\begin{aligned} pu &= \mathbf{pproj} \cdot \mathbf{u} \\ pv &= \mathbf{pproj} \cdot \mathbf{v} \end{aligned} \quad (4.28)$$

Figure 4.5 shows the plane projection of one of the example cylinders from Section 4.1. The circle formed by the projected lateral points is clearly visible. At the same time it is noticeable that some points of the second cylinder are also selected. These are only outliers, but should be taken into account when implementing circle detection. They are represented in small numbers, but should be taken into account when implementing circle detection.

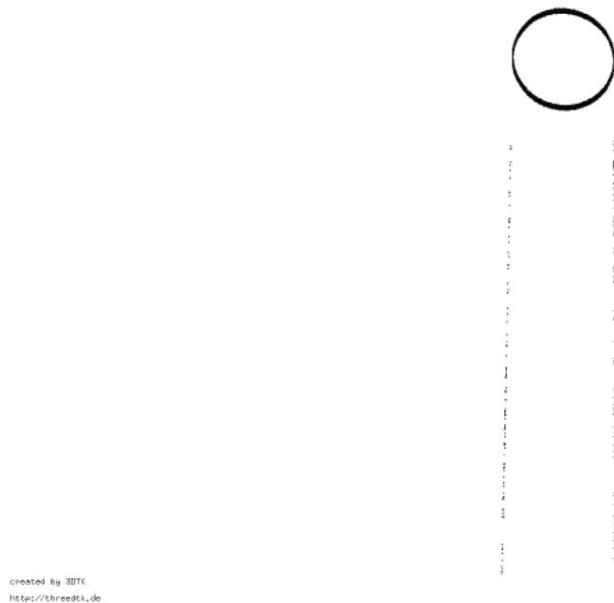


Figure 4.5: Projection of a cylinder along its axis and outliers through a second cylinder perpendicular to the first

4.2.3 Standard Hough transform

As mentioned earlier, [22] uses a standard Hough transform to detect the projected cylinders or circles. For this purpose, an approach according to [13] is implemented. This method will be briefly explained below, followed by a discussion of its weaknesses and suggestions for improvement.

accumulator

Since circle detection involves a three-dimensional Hough transform, the three parameters (radius and circle center (cu, cv)) must be represented and discretized in the accumulator. For this, the user must define the radius range of the cylinders $[rmin, rmax]$ and the resolution of the radius steps r_{sub} before the actual standard Hough transform begins. The following should always hold true: $r_{sub} + 1 = r_{sub}$.

Next, the range of values for u and v in the accumulator must be defined for each ri such that voting is possible for every possible circle with center (cu, cv) in the accumulator plane corresponding to the radius ri . The limits and resolution of u and v can either be predefined by the user or calculated using the projected points and ri . The former has the disadvantage that the user must define a fixed accumulator space for all axes and radii beforehand. This can lead to the accumulator being too large or, in the worst case, too small to find all possible circles.

One way to dynamically define the size of the Hough space is to use projected points. For both directions u and v , the minimum and maximum values of the projected points are recorded, resulting in the value ranges $[umin, umax]$ and $[vmin, vmax]$. The vertices of a rectangular accumulator plane for ri , where all Hough transformations of the projected points are to be represented, can be described by:

$$[umax + ri, vmin \ddot{y} ri] [umax + ri, vmax + ri] [umin \ddot{y} ri, vmin \ddot{y} ri] [umin \ddot{y} ri, vmax + ri] \quad (4.29)$$

This limitation has the disadvantage that the individual levels of the accumulator are of different sizes, which means that either the resolution or the number of sampling points of the individual levels must differ. This is not optimal for the implementation and can be avoided by fulfilling the following condition:

$$\begin{aligned} umax &> cumax \\ umin &< cumin \\ vmax &> cvmax \\ vmin &< cvmin \end{aligned} \quad (4.30)$$

Where $[cumin, cumax]$ and $[cvmin, cvmax]$ describe the maximum and minimum values in $\mathbf{u}$ and $\mathbf{v}$, **respectively**, of all circle centers. If this condition is met, then the vertices of an accumulator plane can be defined as:

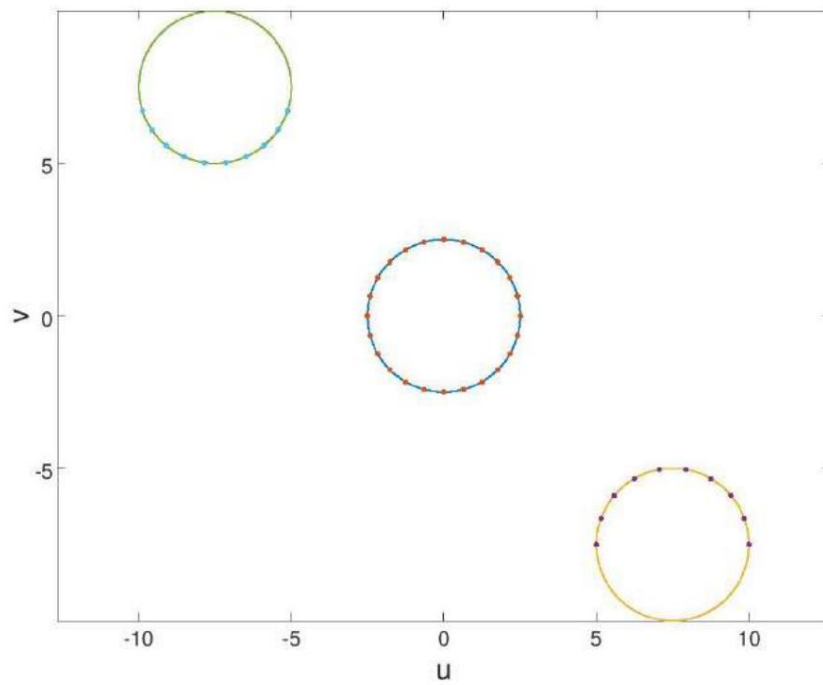
$$\begin{matrix} [umax, vmin] & [umax, vmax] \\ [umin, vmin] & [umin, vmax] \end{matrix} \quad (4.31)$$

An accumulator spanned in this way can no longer completely represent the transformed boundary points, however, all circle centers lie in the Hough space quantized by the accumulator, as long as the condition is not violated.

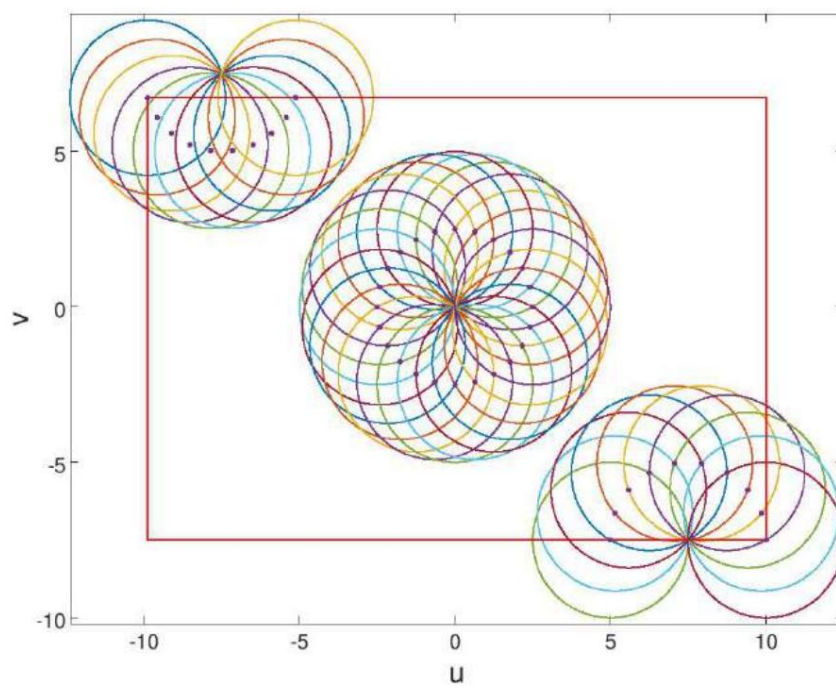
Figure 4.6 shows three example circles in the upper part of the image. While the central circle is fully sampled and thus corresponds to the image presented in [22], only half of the yellow outer circle is sampled, and not even half of the green outer circle. The Hough space, which is possible if condition 4.30 is satisfied, is shown in red. However, this is not the case, since $vmax < cvmax$ and $vmin = cvmin$. The former leads to the fact that the center of the green circle lies outside the Hough space.

The latter leads to a boundary condition, which is why the center of the yellow circle lies on the edge of the accumulator. With cylinder detection, this boundary condition can quickly lead to the circle's center lying outside the accumulator due to the cylinder axis's inherent errors; therefore, it is excluded from the condition.

A major drawback of this method is that even with this simplification, the accumulator structure is still not optimal. This is because not all points from P_c correspond to cylinder points with a suitable axis. Often, some points are selected whose spherical normals happen to align with the cylinder axis direction. These incorrectly selected points from P_c , if they are far from the circles, can unnecessarily expand the accumulator space and significantly degrade the spatial resolution due to the resulting limited sampling.



(a) Sampled circles example



(b) Hough transformations of the circle points and Hough space (red) calculated without radius

Figure 4.6: Representation of sampled example circles and their Hough transformations. Development of a randomized Hough transformation for cylinder detection in laser scans.

Voting

In the case of the standard Hough transform for circle detection, a Hough transform must be determined for each point **P2D**, calculated from the set P_c , analogous to plane detection .

The Hough transform of any point $\mathbf{pk} = (pu, pv)$ corresponds to the Hough transformation of the point $\mathbf{pk} = (pu, pv)$ onto the Hough transform. the accumulator plane corresponding to the radius ri a circle with the circle's center (pu, pv) and the radius ri . Voting is therefore for all cells defined by the function:

$$(ri \cos \tilde{y} + pu, ri \sin \tilde{y} + pv) \quad (4.32)$$

be touched. This voting is repeated for all levels as well as projected points, until Each point on each accumulator level has voted for a circle.

Evaluation of Hough space

After the voting, the maximum peak in Hough space is sought in [22]. This circle corresponds to the position and radius of the most probable cylinder.

The cylinder points are then deleted, and if any further points are present...

are, a new orientation is considered [22]. Possible further peaks, which are from cylinders with originating from the same axis, are processed according to the described circle detection algorithm (page 63 in [22]) not evaluated. However, since cylinders with seemingly the same or at least very similar axes are also found and selected in the real examples, it seems

Several hypotheses for a cylinder axis direction in [22] are permissible and desirable.

This would theoretically be possible due to the local threshold used to evaluate the accumulator. However, it would also mean that for each cylinder in the scene, a

A custom standard Hough transform must be calculated.

4.2.4 Randomized Hough transform

In order to solve or at least partially solve most of the problems described in section 4.2.3

To minimize the error rate, this work avoids using a standard Hough transform for circle detection. Instead, an RHT is used, as is the case for cylinder axis detection.

implemented. This has some major advantages: The RHT can be significantly faster than a Hough transform, which is important because circle detection needs to be calculated very frequently.

Furthermore, the accumulator does not need to be determined before the Hough transform. but can be dynamically built up during RHT.

Another difference in the implementation between the approach from [22] and this variant is evident in the evaluation of the Hough space. Instead of considering only the best maximum of the accumulator, all promising circles are directly validated during the RHT and, if positive, are placed as close as possible to the projected points of the nearest neighborhood using a least-squares approach. Subsequently, only the projected points are deleted, and circle detection continues for the corresponding axis; the original three-dimensional points are not deleted.

It is not possible to delete points from the original point cloud because there can be multiple detections for any two cylinders z_i and z_k if the angle between the cylinder axes w_i and w_k is small. Thus, it is possible to detect z_i as a circle when projected along both w_i and w_k , with the latter leading to a larger error in the cylinder parameters. Therefore, it is not possible to remove points from the point cloud after a successful detection, as a better matching with a different axis may be possible. For this reason, at the end of the algorithm, all multiple detections must be identified and the conflict resolved by deleting the less accurate cylinders.

The following sections describe the dynamically building accumulator, the voting of the RHT, and finally the validation of the promising circles. The previously mentioned least-squares approach for better fitting the circles will be described in the following section 4.2.5, and the intersection detection in the next chapter 4.3.2.

Accumulator and voting

Unlike the accumulator of the standard Hough transform, the accumulator for the RHT does not need to be created at the beginning of the voting process. Instead, the accumulator is built up along with the progressing RHT. The basic principle is as follows: In each RHT iteration, three random points are selected from the set **P2D**. If these are not collinear, a circle **K** with center $cm = (um, vm)$ and radius r is calculated from them, and the three parameters are then rounded to l decimal places. The rounded circle parameters $(u, \tilde{y})$

$m, \tilde{y}_m, r_{\tilde{y}})$ describe a cell in the accumulator. If $r_{\tilde{y}}$ is found, a new entry is created with parameters (u) with a counter value equal to 1. If an entry with the matching parameters already exists, its counter is incremented. [10]

This strategy allows for the creation and incrementing of only the necessary cells in the accumulator during voting. It eliminates the need to deploy the entire accumulator at the outset. This offers the significant advantage of allowing the accumulator size to be expanded, thus preventing false detections due to an undersized accumulator. In addition to size, the accuracy with which the circles are detected can also be adjusted. However, this is not critical for the presented algorithm, as it employs a very coarse, but faster, relative height targeting (RHT). This approach is feasible because subsequently validated circles are further refined using a least-squares method to pinpoint their nearest neighbors.

Validation

Not only does the creation of the accumulator occur at runtime, but also the validation.

Promising circles are already calculated during the RHT. As soon as an entry has a

If the counter is equal to the validation threshold v , the corresponding circle is validated. [10]

For this purpose, all points from **P2D** are selected whose distance to the circle, described by the parameters (u_m, v_m, r) , is smaller than a threshold value $tdis$. $tdis$ should be larger than the inaccuracy arises from rounding. Furthermore, the number of all points will be **Pinlier**. Those located within the circle but at a greater distance than $tdis$ were counted.

Once **Pcirc** is created and the number of **Pinliers** is determined, the validation can begin.

This consists of two steps: a quick, rough pre-sorting and a finer, but

significantly more computationally intensive evaluation. For the former, it is first checked whether the number of **Pcirc** is larger than **Pinlier** to ensure that no random circle is included in a cluster.

is detected from random points. A possible extension of this query can help to

to weed out some very unlikely circles in advance, so that no [something] is possible for them.

A more detailed analysis is required. Two additional queries have been incorporated for this purpose.

Both of these functions operate with a user-defined threshold. Firstly, it is checked whether the number of points **Pcirc** is greater than $tmin$. This threshold tests whether a

The minimum number of points lies on the detected circle. For the second query, the

Cost of the circle, represented by the number of points **Pcirc** divided by r , calculated.

The lower the costs, the greater the probability of drawing a circle at random.

to have fitted the points distribution. For this reason, it makes sense to at least have a small

To set a threshold value $tcost$ for the minimum costs. For the evaluation in Chapter 5,

$tmin = 250$ and $tcost = 5$ were selected. A rough pre-sorting like this takes up time due to the

A very small chosen threshold value has no influence on the result of the circle detection, since in this way improbable circles are also filtered out by the second validation step.

However, it is significantly faster and can thus help to speed up circle detection somewhat.

For the second validation step, the circle is divided into $nseg$ equal-sized circular sectors.

The length of the circular arc b should always be defined by the user, while the angle

The γ of the circular sector in radians changes depending on r . Since the length of the circular arc is defined as:

$$b = 2\gamma \cdot r \Rightarrow \gamma = \frac{b}{2r} \quad (4.33)$$

γ can be described as a function of r and b :

$$\gamma = r \frac{b}{2} \quad (4.34)$$

Thus, the number of circular sectors $nseg$ as a function of r and b is:

$$nseg = \text{ceil}\left(\frac{2\gamma}{r}\right) = \text{ceil}\left(\frac{2\gamma}{r}\right) \quad (4.35)$$

To ensure that a circle is divided into enough sectors despite a very small radius, if $r_{\min} < r_{\min} = 1$, then $r_{\min} = r_{\min}$. Thus, for $r_{\min}$:

$$n_{\text{seg}} = \text{ceil}(2\sqrt{b}) \quad (4.36)$$

A sector s_i is considered positive as soon as a point from **Pcirc** lies on it [10]. For successful validation, at least $p_{\text{seg}}\%$ of the sectors must be positive [10].

If the number of positive sectors is too low, the validation is aborted with a negative result. To avoid further validation of the circle with the parameters (u, r) , the corresponding entry in the accumulator is marked as validated, but the circle is not added to the list of found circles.

A validation is positive as soon as at least $p_{\text{seg}}\%$ of the sectors are positive. The previously roughly determined circle with the parameters (u, r) is fitted to the points **Pcirc** as closely as possible using a least-squares approach (described in Section 4.2.5). Subsequently, all entries that intersect the newly calculated circle **cLS** are removed from the accumulator, since in most cases there are many entries there that are very similar to the detected circle [10].

For the evaluation in Chapter 5, $b = 0.015$ and $p_{\text{seg}} = 50\%$ are used for the detection of solid cylinders and $p_{\text{seg}} = 25\%$ for the detection of half cylinders. Section 5.3 also shows that a p_{seg} that is too small can lead to a rapidly increasing false detection rate. The same applies analogously to a b that is too large. At the same time, a p_{seg} that is too high or a b that is too small means that cylinders with only a few points on their surface cannot be validated (see Section 5.3 for an example).

4.2.5 Least-Square Circle Fitting

To then position the validated, roughly detected circles of the RHT as accurately as possible in the immediate vicinity of the points, a least-square method is used. The approach described in [25] was adopted for this purpose. The two-dimensional points **Pcirc**, selected from **P2D**, serve as the input point set for the algorithm. They define the circle **K** with center point (x, y) and radius R , where $\ddot{y} = [(x_i - x)^2 + (y_i - y)^2]$.

The error is defined as the difference between the constant area $\ddot{y}R^2$ and the area of the circle with radius R centered at $\bar{x}$ and $\bar{y}$:

$$(x_i - \bar{x})^2 + (y_i - \bar{y})^2 - \ddot{y} \quad (4.37)$$

the cumulative error is given by:

$$x, \bar{y} = \ddot{y} \left[\sum_{i=1}^N \frac{1}{\ddot{y}} \right] e(R)^2 \quad (4.38)$$

or

$$J = \frac{e}{2} = \sum_{i=1}^N [R^2 - \tilde{y}_i]^2 \quad (4.39)$$

For the best possible fit, $J(R, x, y)$ must be minimized. This minimization can be calculated using partial derivatives. Differentiating J partially with respect to R yields:

$$\frac{\partial J}{\partial R} = 2 \sum_{i=1}^N [R^2 - \tilde{y}_i](2R) = 0 \quad (4.40)$$

which in turn leads to the following:

$$NR^2 = \sum_{i=1}^N \tilde{y}_i \quad (4.41)$$

The partial derivative with respect to $\tilde{x}$ yields:

$$\frac{\partial J}{\partial \tilde{x}} = 0 = 2 \sum_{i=1}^N [R^2 - \tilde{y}_i](\tilde{x}_i - \tilde{x}) \quad (4.42)$$

or

$$\sum_{i=1}^N [R^2 - \tilde{y}_i] \tilde{x}_i = \sum_{i=1}^N [R^2 - \tilde{y}_i] \tilde{x} = 0 \quad (4.43)$$

which follows from:

$$R^2 \sum_{i=1}^N \tilde{x}_i = \sum_{i=1}^N \tilde{x}_i^2 \quad (4.44)$$

Similarly, the following applies to the partial derivative for $\tilde{y}$:

$$R^2 \sum_{i=1}^N \tilde{y}_i = \sum_{i=1}^N \tilde{y}_i^2 \quad (4.45)$$

Assuming the **Pcirc** are error-free, then for every point $(\tilde{x}_i, \tilde{y}_i)$ on the circle the following holds: Equation:

$$\tilde{y}_i = R^2 - \tilde{x}_i^2 \quad (4.46)$$

These partitions have the advantage that, despite their quadratic nature, they can be solved = $N x^3$.

is introduced that

$\tilde{x}_i = \tilde{y}_{i=1} \wedge \{x_i\}, \tilde{y}_{i=1} \wedge \{y_i\}$ For this purpose, the notation $\tilde{x}_i = \tilde{y}_{i=1} \wedge \{x_i\}, \tilde{y}_{i=1} \wedge \{y_i\}$ etc. Thus, equation 4.41 yields:

$$NR^2 = \sum_{x^2} -2 \sum_x \tilde{x} + N \tilde{x}^2 + \sum_{y^2} -2 \sum_y \tilde{y} + N \tilde{y}^2 \quad (4.47)$$

From equation 4.44:

$$R^2 - \sum_x = \sum_{x^3} -2 \sum_{x^2} \tilde{x} + \sum_x \tilde{x}^2 + \sum_{xy^2} -2 \sum_{xy} \tilde{y} + \sum_x \tilde{y}^2 \quad (4.48)$$

4.2. Position and radius calculation

41

and analogously for equation 4.45:

$$R^2 - \bar{y} = \bar{x}^2 - 2 \bar{x} \bar{y} + \bar{y}^2 + \bar{y}^2 - 2 \bar{y} \bar{y} + \bar{y}^2 \quad (4.49)$$

If equation 4.47 is now multiplied by N , the following results:

$$\ddot{y}_x \ddot{y}_x N \ddot{y}_x^2 - x^2 \ddot{y}_x^2 - N \ddot{y}_x^2 + \ddot{y}_x^2 \ddot{y}_x^2 - 2 \ddot{y}_x \ddot{y}_x N \ddot{y}_x^2 - y^2 \ddot{y}_x^2 = 0 \quad (4.50)$$

Similarly, you can also use 4.47 with y . Multiply and subtract N times 4.44:

$$\ddot{y}_x \ddot{y}_y - N \ddot{y}_x^2 - 2 \ddot{y}_x \ddot{y}_y + \ddot{y}_y^2 - N \ddot{y}_y^2 = 0 \quad (4.51)$$

From 4.50 and 4.51, the solution for $\bar{x}$ and $\bar{y}$ is obtained :

$$\bar{x} = \frac{a_1 b_2 - a_2 b_1}{a_1^2 + b_1^2} \quad (4.52)$$

where

$$\begin{aligned} a_1 &= 2(\ddot{y}_x \ddot{y}_x - N \ddot{y}_x^2) & b_1 &= 2(\ddot{y}_x \ddot{y}_y - N \ddot{y}_x \ddot{y}_y) \\ a_2 &= 2(\ddot{y}_x \ddot{y}_y - N \ddot{y}_x \ddot{y}_y) & b_2 &= 2(\ddot{y}_y^2 - N \ddot{y}_y^2) \end{aligned} \quad (4.53)$$

$$\begin{aligned} c_1 &= (\ddot{y}_x^2 - N \ddot{y}_x^2 + \ddot{y}_x^2 \ddot{y}_x^2) \\ c_2 &= (\ddot{y}_y^2 - N \ddot{y}_y^2 + \ddot{y}_y^2 \ddot{y}_y^2) \end{aligned} \quad (4.54)$$

This leads to the solution for the center of the circle:

$$\bar{x} = \frac{c_1 b_2 - c_2 b_1}{a_1 b_2 - a_2 b_1} \quad (4.55)$$

$$\bar{y} = \frac{a_1 c_2 - a_2 c_1}{a_1 b_2 - a_2 b_1} \quad (4.56)$$

With these, it is possible to calculate using a substitution in 4.47 R^2 :

$$R^2 = \frac{1}{N} (\bar{x}^2 - 2 \bar{x} \bar{y} + \bar{y}^2 + \bar{y}^2 - 2 \bar{y} \bar{y} + \bar{y}^2) \quad (4.57)$$

In this way, the best possible circle fit can be calculated from the roughly detected, validated circle of the RHT and the points from its immediate vicinity. [25]

Even though such a least-squares approach is theoretically unnecessary, since a more accurate RHT can provide a better hypothesis for circle detection, combining these two algorithms saves time and simultaneously calculates a good circle estimate. Thus, a least-squares approach contributes to a better-fitted cylinder.

4.2.6 Pseudocode of RHT circuit detection

Pseudocode 4 summarizes the most important steps of circle detection. This pseudocode provides a general overview of circle detection; the details of...

Individual functions, such as validation, are discussed in the sections above.

Algorithm 4 Circle Detection - RHT

Require: Input: **P2D**, possible cylinder points projected along cylinder axis

```

1: Accumulator A= {}
2: District list VK= {}
3: while t < n do
4: Choose p1, p2, p3  $\ddot{y}$  P2D randomly
5:   if Collinear(p1, p2, p3) then
6:     t++;
7:     continue;
8: else
9:   t++;
10:  Circle K = calculateCircleFrom3Points(p1, p2, p3)
11:  if A.hasEntry(K) then
12:    Accumulator cell Z = A.getEntry(K)
13:    Z.counter += 1
14:    if Z.counter >= v and Z.validated == false then
15:      isCircle = validate(ZK)
16:      Z.validated = true
17:      if isCircle then
18:        KLS = LeastSquare(ZK)
19:        VK.add(KLS)
20:        A.deleteCutCircles(KLS)
21:        t = 0;
22:      end if
23:    end if
24:  else
25:    A.add(K, counter=1, validated = false)
26:  end if
27: end if
28: end while
29: return VK

```

4.3 Cylinder end detection and validation of detected cylinders

Using cylinder axis detection and circle detection, it is possible to determine all 5 cylinder parameters, where the position of the cylinder is only given by a point on the axis. However, no solution is shown in [22] for subsequently calculating the cylinder length.

In this work, the point on the cylinder axis is used to locate the starting and ending points of the axis. The points on the circles found are not used for this purpose, since two cylinders with the same axis and radius, separated only by a translation along the axis, will project the same circle.

After the algorithm has calculated the last cylinder ends, all detected cylinders must then be compared and duplicates removed. This is not necessary in [22], since a separate circle detection is calculated for each detected cylinder axis, which only evaluates a maximum and then deletes the cylinder points from the point cloud. Because no points are deleted from the point cloud in the implemented case, it is possible to find multiple detections. These detections must be identified and ultimately evaluated so that only the best possible fit is returned at the end of the cylinder detection. Therefore, the following sections first explain cylinder end detection (see 4.3.1) and then the detection of duplicate cylinders (see 4.3.2).

4.3.1 Cylinder end detection

Cylinder end detection occurs directly after circle detection. This has the advantage that the entire point cloud does not need to be considered, but only the point set P_c , already pre-selected for circle detection, whose spherical normals represent the cylinder axis. The points of a circle are not used directly, as a circle can also represent two cylinders with the same axis and radius.

To determine all endpoints and starting points of the cylinder axis $\mathbf{w}$ from P_c , all points from P_{cyl} that lie on the cylinder's lateral surface must be selected for each cylinder $\mathbf{Z}$. For this purpose, a straight line $\mathbf{g}$ is defined from $\mathbf{w}$ and the point on the axis p_w , given by the transformed center of the cylinder's circle $\mathbf{K}$, and the distance d to $\mathbf{g}$ is calculated for each point $p \in P_c$. All N points for which $r - \tilde{y} \leq d \leq r + \tilde{y}$, where r is the radius of $\mathbf{K}$ and $\tilde{y}$ is a small threshold value, form P_{cyl} .

Next, all points from P_{cyl} are projected perpendicularly onto $\mathbf{g}$ and sorted such that p_1 and p_N have the greatest distance between any two points in P_{cyl} projected onto $\mathbf{g}$, where p_1 and $p_N \in P_{cyl}$. Then, the distance d_n between each projected point p_i and its neighbors p_{i+1} and p_{i-1} , if any, is calculated. The mean d_{mean} is then calculated from all distances d_n . Since the points are arranged along $\mathbf{g}$ due to the sorting, d_{mean} provides a rough indication of the average distance between any two points on the line.

For this reason, d_{mean} can be used to divide all projected points into groups, where the distance between each point p_i from any group $\mathbf{G}_k$ and

Its possible neighbors p_{i+1} , p_{i-1} must be less than or equal to $d_{mean} \cdot tm$. The factor tm , multiplied by d_{mean} , allows for adjusting the mean distance d_{mean} . The group with the most elements is called the lateral surface area of the cylinder, **PMZ**. Due to the pre-sorting, the beginning and end of the cylinder axis can be directly determined from the mean of the *first* and last points.

The advantage of this approach lies in its simple implementation and the pre-selected point set P_c , which is usually significantly smaller than the point cloud. However, this also has the disadvantage that not all cylinder points are present in P_c . Specifically, the spherical normals of the points on the cylinder's lateral surface often differ too greatly from those of the other lateral surface points, preventing them from voting for the origin plane with the plane vector equal to the cylinder axis. Consequently, the calculated length of the cylinder, defined by the cylinder axis endpoints, always exhibits a systematically negative error. If such an error is critical, the cylinder endpoint calculation must be adjusted, or an improvement to the cylinder endpoints must be implemented after the algorithm is returned.

The cylinder lateral points can be found, for example, by searching for a point cloud along the cylinder axis; by subsequently projecting the points onto the cylinder axis, new maxima or minima for the cylinder end and cylinder start can be defined.

4.3.2 Detection of duplicate cylinders

The final step of cylinder detection is validation of the detected cylinders. This involves identifying cylinders found multiple times and subsequently discarding the weaker fits. This is necessary because several similar cylinder axis hypotheses can be calculated, potentially leading to cylinders being projected multiple times onto a plane and detected as erroneous circles. Any cylinder Z_i is only compared to another cylinder Z_j if the angle between the two cylinder axes w_i and w_j is less than tw . For the experiments in Chapter 5, tw was chosen to be less than 5° . Furthermore, if the minimum distance d_{min} between the two lines defined by w_i , the cylinder axis starting point psi_i , and endpoint pei (or w_j , psj , and pej , respectively) is less than the sum of the radii r_i and r_j , the two cylinders intersect and are considered identical detections. If many cylinders with similar or identical axes are very close together, the required minimum distance d_{min} between the cylinders should be replaced by $r_i + r_j \cdot \gamma_{tdis}$ to avoid filtering out two neighboring cylinders. This is not necessary for the experiments described in Chapter 5.

After identifying two identical cylinders, the likely inferior one must be removed from the resulting cylinder set **Z**. This is done by comparing the points on the cylinder's lateral surface **PMZ** and deleting the cylinder with the fewer points. If two cylinders have the same number of points, the cylinder with the larger radius is deleted. For a more precise cylinder length determination, such as one using all points, the radius should be replaced by the lateral surface area.

Subsequently, all detected cylinders are assigned the cylinder axis, the radius, the calculated point on the cylinder axis, and the resulting calculated cylinder axis ends.

returned to the user. In the following sections, cylinder detection is evaluated using simulation and real point clouds, and the individual errors of the 5 RHT are discussed.

Parameters calculated.

Chapter 5

Experiments and evaluation

As described in Chapter 4, cylinder detection consists of two sequential steps: cylinder axis detection and circle detection. In addition to these main steps, the cylinder length is calculated using the average distance between the points on the cylindrical surface. Furthermore, identical cylinders are identified at the end of the detection process, and only the best match is retained. Three CAD models were used for the evaluation of this cylinder detection (see Appendix 6 for illustrations of the models), two of which were 3D-printed to capture the real-world data. Therefore, the cylinders are relatively small, with radii ranging from approximately 2.5 mm to 10 mm.

With two exceptions, the parameters for evaluating cylinder detection were always chosen to be the same. For example, an accumulator with 636,473 square cells was used for detecting the cylinder axes. To subsequently determine the accumulator's maxima, a 20x20 square window was used for the sliding window function. The maximum number of detected cylinder axes, which is passed on to circle detection, is one of the two variable parameters. For all experiments, this corresponds to the number of unique cylinder axes in the model multiplied by a factor of 10. A validation threshold of 3 was used for the relative height of the circle detection.

If a circle is found three times, it is tested whether it has at least 250 points and its cost is greater than or equal to 5. If this is the case, in the second validation step, the circles are divided into sectors, with each sector having an arc length of 0.016 mm. For successful validation, at least $pseg\%$ of sectors must be positive. $pseg\%$ is the second variable parameter. For the detection of cylinder models with full cylinders, $pseg\%$ was set to 50%, while for the detection of partial cylinders, $pseg\%$ is set to 25%. After cylinder detection, multiple detected cylinders are searched for, and potentially weaker detections are deleted so that only one identical cylinder is returned to the user. Two detected cylinders are considered identical as soon as there is an angle $\gamma \leq 5^\circ$ between the cylinder axes and the two cylinders intersect.

While the RHT for cylinder axis detection is similar to the standard Hough transform described in [22], a different algorithm was chosen for circle detection.

The RHT algorithm developed in this work comprises the RANSAC approach from paper [15], combined with the basic RHT concept described in [10] and a least-squares optimization based on [25]. Due to the combination of three approaches, the basic functionality of circle detection is first demonstrated using a simple simulated example, before the actual validation of cylinder detection and subsequently of partial cylinder detection on simulated and real data is carried out.

5.1 Evaluation of the circle detection

To test the basic functionality of circle detection as efficiently as possible using a cylinder example, the maximum number of cylinder axes is minimized to one. This makes it possible to directly compare the detected circle centers with absolute values and simultaneously consider a selection of different radii and cylinder heights.

For this reason, a total of 16 cylinders are considered for the evaluation of circle detection, all standing vertically in a 4x4 matrix on a base plate. There is a distance of 30 mm between each cylinder and its neighbors. Four different radii (2.5 mm, 5 mm, 7.5 mm, 10 mm) and four different heights (10 mm, 15 mm, 20 mm, 25 mm) are modeled using the CAD software Onshape [3]. Figure 6.1 shows the model from different perspectives and with the most important dimensions.

The 3D model is subsequently exported as a mesh for cylinder detection and sampled using CloudCompare [2]. The advantage of such a point cloud is that the absolute positions of the individual cylinders within the point cloud are known. Although a base plate was used for the modeling, it must be removed from the point cloud because planes can complicate or prevent cylinder detection [22]. One possible solution to this problem is, for example, a preceding plane detection. The section on evaluating cylinder detection (5.2) discusses the problem of plane points and how they can complicate or prevent cylinder detection in more detail. After removing the plane, the point cloud for testing circle detection consists of 44,772 points. The normals required for cylinder detection, as well as the display of point clouds, Hough spaces, and cylinders, are calculated using the 3DTK toolkit [1].

Since a randomized procedure is used for circle detection, the results of 1000 detections were considered for the evaluation. Therefore, in a test dataset, each cylinder should be detected 1000 times, and ideally, 16000 cylinders should be found.

In the specific case of circle detection validation, a total of two test datasets were generated. In the first attempt, the circle detection was given a perfect cylinder axis, while in the second attempt, a detected and therefore erroneous cylinder axis was used. This division is necessary for an initial evaluation because an erroneous axis leads to a systematic error in the projection, which cannot be corrected by the least-squares approach.

5.1.1 Evaluation of circle detection with a fault-free cylinder axis and noise-free point cloud

To evaluate the basic functionality of circle detection, calculating the cylinder axis detection is unnecessary; instead, the cylinder axis is directly passed to the circle detection. In the case of a right-handed coordinate system, the axis would be $[0 \ 0 \ 1]$, but since 3DTK uses left-handed coordinate systems by default, the cylinder axis is $[0 \ -1 \ 0]$, or, due to the possible inversion, $[0 \ 1 \ 0]$. Such a test of the basic functionality is necessary to observe whether the algorithm, combining the approaches from papers [15], [10], and [25], functions flawlessly and robustly, or whether it exhibits a systematic error. When using detected axes with errors, no direct statement can be made about the functionality of circle detection, as the error propagates through the projection of the 3D points along the cylinder axis onto a plane.

Considering the results of the circle detection, with a given axis and 1000 randomized trials, all cylinders were found exactly 1000 times. Consequently, all circles were at least detected. For this reason, the following sections will focus on the errors of the radii in relation to the size and then analyze the absolute positions of the circle centers and their errors.

Radius determination errors

At the start of the analysis, all detected cylinders are sorted so that the 16,000 detections are divided into 16 sets of 1,000 detected circles each. Then, the detected radii for each cylinder are examined individually. The model contains four different radius sizes: 2.5 mm, 5.0 mm, 7.5 mm, and 10.0 mm. Each radius appears exactly four times in the model, and no two cylinders with the same radius have the same height. Therefore, a total of 4,000 circles must be found for each radius size in 1,000 iterations. Figure 5.1 shows all calculated radii in a histogram. It is clearly visible that there are four peaks at 2.5 mm, 5.0 mm, 7.5 mm, and 10 mm. Each of these radii was detected exactly 4,000 times.

Due to the small differences between the individual detected radii, it is not possible to represent them in a histogram with a radius scale of 2 to 10 mm. Therefore, the errors of the radii are always considered in millimeters. In the four histograms shown in Figure 5.2, the errors of the radii are broken down by radius size, each in its own histogram. It is noticeable that in the sub-histograms, exactly four maxima are recognizable for each radius, with the exception of the histogram for the radius of 7.5 mm. The histogram for the radius of 2.5 mm, shown in Figure 5.2(a), is considered as an example. The maxima of the errors in this case are -0.00041, -0.0004, -0.00039, and -0.00036 mm. Each deviation is detected exactly 1000 times. A similar distribution of maxima is also found at radii of 5.0 mm and 10 mm. The only exception is the error at a radius of 7.5 mm, shown in 5.2(c), where...

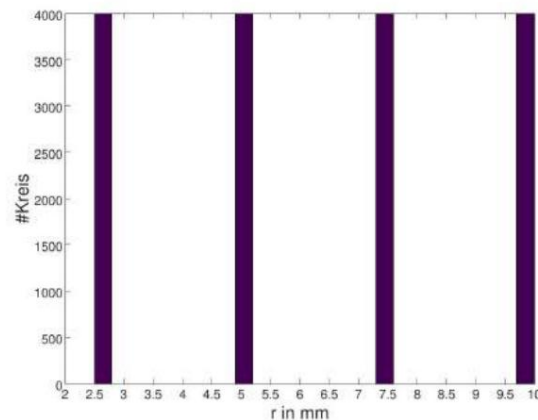


Figure 5.1: Histogram of the found radii

Only three peaks were found in total, however the first peak at -0.0119mm has twice the Height. In the case of a radius of 7.5 mm, two error peaks coincide.

One explanation for this deterministic distribution can be found in the design of the model.

again. Although exactly four cylinders always have the same radius, their height and The position is always different. The differences are in height and therefore in number and

The distribution of the surface area points is reflected in the detection of the circles. Due to

At first glance, such deterministic behavior seems unusual to the RHT.

However, since after successful circle detection, a least-squares approach across all points

The calculation is performed in the nearest circle neighborhood ($r_{min} = rrht - 1$, $r_{max} = rrht + 1$) according to the randomized detection provides a deterministic fitting in a small environment. This combined approach

Despite very fast, coarse RHT, this approach allows for a very good circle to be reliably achieved.

detect. This deterministic improvement also leads to the four clearly identifiable

Peaks in the histograms. The error range of the individual radii is so small that

A successful radius determination has clearly been achieved.

Radius [mm]	mean Radius [mm]	Mean error [%] depending on the radius	Standard deviation [mm]
2.5	2.4996	-0.0156	1.8711e-05
5.0	4.9992	-0.0155	1.4792e-05
7.5	7.4988	-0.0157	1.2992e-05
10.0	9.9984	-0.0159	1.4792e-05

Table 5.1: Mean error and standard deviation of the four radii

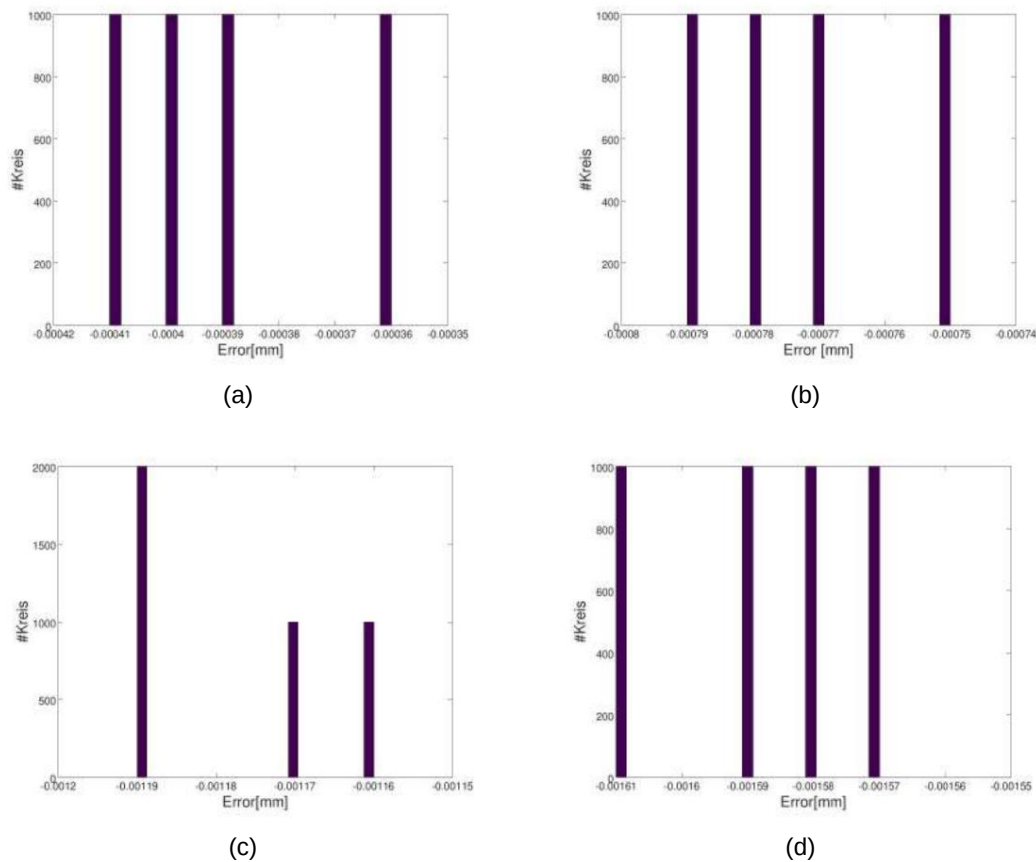


Figure 5.2: Errors of the four radii: (a) Errors of the circles with radius 2.5 mm; (b) Errors of the circles with radius 5.0 mm; (c) Errors of the circles with radius 7.5 mm; (d) Errors of the circles with radius 10 mm.

Considering the error range for a radius of 2.5 mm, for example, it lies between -0.00036 mm and -0.00041 mm, while the errors for circles with a radius of 10.0 mm range from -0.00157 mm to -0.00161 mm. Calculating the mean error for each radius, the values $-3.900\text{e-}04$, $-7.725\text{e-}04$, $-11.775\text{e-}04$, and $-15.875\text{e-}04$ clearly show that the mean error increases with increasing radius. Table 5.1 displays the mean errors, arranged by increasing radius, as well as the standard deviation and the mean error as a function of radius. It is noticeable that for all four radii, this value is approximately -0.016%.

Furthermore, it should be noted that all the radii found are minimally smaller than the absolute value and therefore always have a negative error. There are two possible sources of error for this. Firstly, the systematically negative error can be attributed to the least-squares fitting at the end of a successful RHT (Reference Height Technique). Secondly, it is possible that this minimal deviation actually exists because the point cloud is generated by sampling a 3D model. Cylinders and circles must be represented by polygons in CAD programs.

Due to the necessary approximation, this minimal deviation can also occur. In the following, this error is neglected due to its small size and imperceptible impact.

In summary, the evaluation of the radius detection for a precise cylinder axis shows that all radii were found and lie within a very good error range, with a very small data spread. The only noteworthy finding is the very small, systematically negative error of the radius. However, these deviations are so small compared to the later errors in the actual data or the inaccuracies caused by plane points that they have no impact later on. Overall, it can be stated that the cylinder parameter radius was consistently detected very accurately.

Position determination errors

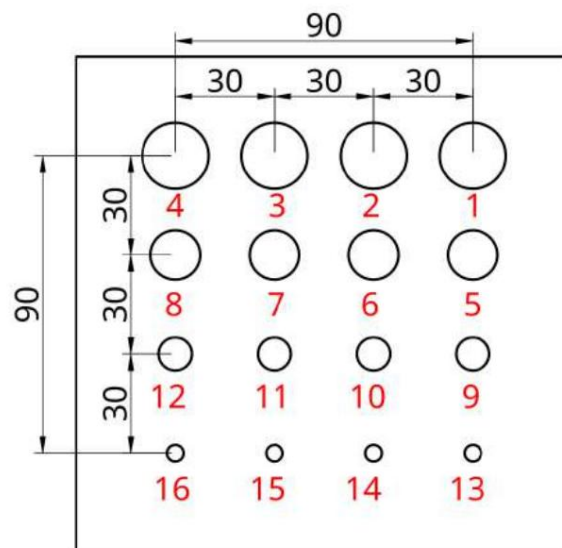


Figure 5.3: Representation of the cylinder numbers (red) in model 1

In addition to the radius, the cylinder's position in three-dimensional space is also determined through circle detection. This is achieved by transforming the projected circle center point back into three-dimensional space. For evaluating the axis point, the 3D point is projected back onto the base plate, allowing for direct comparison with the absolute circle center positions. This is only possible because, in this specific case, the data set is simulated with a known cylinder axis.

For the absolute positions, the origin [0,0] of the projected plane axes u and v was placed in the center of the base plate. The cylinders are all perpendicular to this plate, so that the lateral surfaces of a cylinder, projected along their axis, form a circle on the plane. The absolute center of the circle of the cylinder with radius 10.0 mm and height 25 mm

lies at [-45, -45], while the center of the cylinder has a radius of 2.5mm and a height of 10mm. at [45, 45]. The cylinder numbers used below are shown in Figure 5.3. depicted.

Cylinder given by [Radius, height]	Optimal position [u,v] [mm]	Absolute mean Error[u,v] [mm]	Distance Standard deviation [mm]
Cyl. 1 [10mm, 25mm] [-45 -45]		[0,0 0,0]	0.0
Cyl. 2 [10mm, 20mm] [-15 -45]		[0,0 0,0]	0.0
Cyl. 3 [10mm, 15mm] [15 -45]		[0,0 0,0]	0.0
Cyl. 4 [10mm, 10mm] [45 -45]		[0,0 0,0]	0.0
Cyl. 5 [7.5mm, 25mm] [-45, -15]		[0,0 0,0]	0.0
Cyl. 6 [7.5mm, 20mm] [-15, -15]		[0,0 0,0]	0.0
Cyl. 7 [7.5mm, 15mm] [15, -15]		[0,0 0,0]	0.0
Cyl. 8 [7.5mm, 10mm] [45, -15]		[0,0 0,0]	0.0
Cyl. 9 [5.0mm, 25mm] [-45, 15]		[0,0 0,0]	0.0
Cyl. 10 [5.0mm, 20mm] [-15, 15]		[0,0 0,0]	0.0
Cyl. 11 [5.0mm, 15mm] [15, 15]		[0,0 0,0]	0.0
Cyl. 12 [5.0mm, 10mm] [45, 15]		[0,0 0,0]	0.0
Cyl. 13 [2.5mm, 25mm] [-45, 45]		[0,0 0,0]	0.0
Cyl. 14 [2.5mm, 20mm] [-15, 45]		[0,0 0,0]	0.0
Cyl. 15 [2.5mm, 15mm] [15, 45]			0.0
Cyl. 16 [2.5mm, 10mm] [45, 45]			0.0

Table 5.2: Mean error and standard deviation of cylinder positions

If the absolute error of the positions, as well as the standard deviation, is considered (see Table 5.2) shows that the centers of the circles always have the same exact value. exhibit. For the sake of readability, only two significant figures were shown in Table 5.2. Instead of the calculated five, five are given, as no deviation is apparent in these either. Accurate determination of the circle center, despite fast but inaccurate RHT, is achieved through the Least Square method. Possible approach. Individual histograms will be analyzed based on the result in this case. abstained.

Thus, it can be clearly summarized that the circle centers in simulated Data and perfect axis can be determined very accurately. On the evaluated five No errors can be detected at significant points.

Evaluation of radius and position determination

In summary, it can be clearly stated that circle detection is functional when an error-free axis and a noise-free point cloud sampled from a mesh are used as input. Even cylindrical circles with very few lateral surfaces, represented by a cylinder with a radius of 2.5 mm and a height of 10 mm, are always detected. However, since an error-free axis is an absolute edge case that rarely occurs in reality, the following section tests the functionality of circle detection with an error-prone detected axis. Subsequently, the complete cylinder detection is evaluated using two examples, each with simulation and real-world data.

5.1.2 Evaluation of circle detection with a faulty cylinder axis and a noise-free point cloud

In this section, circle detection is applied to projected plane points that have a systematic error due to the detected, faulty cylinder axis.

This systematic error depends on the size of the error angle between the absolute and detected cylinder axes. For the detection of circles with systematic errors, the following applies: the larger the error and the fewer lateral surfaces a cylinder has, the more difficult it is to detect its circular shape.

In the evaluation of this test set, a case arises that illustrates this point precisely: instead of 16,000 cylinders, only 15,999 cylinders were found. When the detections are distributed among the individual cylinders, it quickly becomes apparent that in 1,000 trials, a cylinder with a radius of 2.5 mm and a height of 10.0 mm was missed exactly once. The reason for this is that it has the fewest lateral surface points, making it significantly more difficult to detect the cylinder even with a slightly shifted axis. The crucial limiting factor here is not the radius of the cylinder (RHT), but rather the validation function.

As described in Chapter 4 (Implementation), the validation function divides a circle into sectors, depending on a user-defined resolution and radius. For the detection of entire cylinders, at least 50% of all sectors must be positive, meaning they must contain a point lying on the circle's edge. If an imperfect cylinder axis is used for the projection, the elliptical projection can result in too few points being located in the immediate vicinity of the circle's edge. This leads to a negative validation. The circle is rejected, and another similar circle must be detected at least v times for further validation. Therefore, cylinders with a small lateral surface area are more prone to false detections.